LEÇON PRATIQUE · JAVA 21 / 25 · MISE À JOUR NOVEMBRE 2026

Aléatoire en Java

Tirer un entier de 1 à 99, mélanger une liste, générer des caractères, reproduire une partie au hasard — avec les générateurs du JDK, pas contre eux.

Public	Développeuse / développeur Java qui doit produire un tirage, un mélange ou une chaîne aléatoire : jeu, loterie, test, génération de jeton. Niveau intermédiaire.
Prérequis	Un JDK 21 ou 25, un terminal, savoir compiler un fichier `.java`. Aucune bibliothèque externe : tout tient dans `java.base`.
Durée	≈ 1 h 45 de lecture + 45 min de reproduction des exemples + 45 min d'exercices (chapitre 11).
Objectif final	Répondre sans hésiter à : « pourquoi mon tirage donne-t-il 100 au lieu de 99 ? », « comment mélanger une liste que je ne peux pas modifier ? », « comment rejouer exactement la même partie ? », « puis-je utiliser `Random` pour un jeton de session ? ».

nextInt · bornes incluses/exclues | Collections.shuffle · Fisher-Yates | char · codePoint · Unicode | graine · SplittableRandom · LXM | SecureRandom · jetons | pondérations · réservoir | Javadoc JDK 25 · ThreadLocalRandom

Les trois questions de départ, et leurs réponses courtes

« Comment tirer un entier de 1 à 99 ? » **`1 + generateur.nextInt(99)`**, parce que `nextInt(b)` va de 0 inclus à `b` exclu. Écrire `nextInt(100)` donne 0, et `Math.round(nextDouble() * 99) + 1` donne 100 une fois sur 200 (chapitre 2).

« Comment mélanger une liste ? » **`Collections.shuffle`** sur une liste modifiable : la liste est modifiée sur place, et `List.of` lève `UnsupportedOperationException`. Le tri doit utiliser l'algorithme de Fisher-Yates, pas un échange avec un indice quelconque (chapitres 4 et 5).

« Et pour des caractères ? » **Un alphabet sous forme de `String`**, puis `alphabet.charAt(nextInt(alphabet.length()))` : cela gère aussi les accents, et se généralise aux emojis via les points de code (chapitre 6).

Sources principales : Javadoc du JDK 25 — `Random`, `RandomGenerator`, `RandomGeneratorFactory`, `SplittableRandom`, `ThreadLocalRandom`, `SecureRandom`, `Collections`, `List`, `Character`, `String`, `Normalizer`, `Collator` · RFC 4086 (Randomness Requirements for Security) · FIPS 140-2 §4.9.1. Toutes les sorties ont été exécutées sur Temurin 25.0.5, en novembre 2026.

Sommaire

1 Le modèle mental : un état, une fonction, une séquence 4
- 1.1 Ce que « aléatoire » veut dire dans le JDK 4
- 1.2 Les cinq classes à connaître, et quand choisir laquelle 4
- 1.3 La règle centrale : un tirage se décrit par son intervalle 5
- 1.4 La grille de décision, en une page 6

2 Tirer un entier : de 1 à 99 et au-delà 7
- 2.1 `nextInt(b)` va de 0 inclus à `b` exclu 7
- 2.2 Les quatre variantes, et l'intervalle réel de chacune 7
- 2.3 La formule générale, et l'intervalle trop large 8
- 2.4 Modulo, arrondi : les deux pièges qui produisent une valeur hors bornes 8
- 2.5 Une classe `Intervalle` qui ne peut pas se tromper 9

3 Long, double, float : les autres bêtes 11
- 3.1 `nextLong` : le même contrat, en 64 bits 11
- 3.2 `nextDouble` : 53 bits, et la borne haute exclue 11
- 3.3 `nextFloat` : 24 bits, ce qui se voit 12
- 3.4 `Math.random()` : ce qu'il cache 12

4 Mélanger une liste : `Collections.shuffle` 14
- 4.1 Le cas normal, et les listes qui refusent 14
- 4.2 La liste est modifiée : copie défensive obligatoire 14
- 4.3 Trois appels de suite : ce que fait vraiment `shuffle` 15
- 4.4 L'équité du mélange, mesurée sur 600 000 essais 15

5 Tirer sans remise : échantillons et pioches 17
- 5.1 Quatre approches, et la bonne selon la taille 17
- 5.2 Le shuffle naïf, ou pourquoi Fisher-Yates existe 17
- 5.3 Le coût du rejet quand on approche du stock 18
- 5.4 Le réservoir : tirer dans un flux qu'on ne peut pas relire 19

6 Caractères et chaînes aléatoires 20
- 6.1 L'alphabet est la bonne abstraction 20
- 6.2 De `'a'` à `'z'` : trois écritures et deux pièges 20
- 6.3 Accents : `char` suffit, `codePoint` pour le reste 21
- 6.4 Emojis et hors BMP : `length()` n'est pas un nombre de caractères 21
- 6.5 Chaînes pondérées, alphabets sans ambiguïté 22

7 Reproductibilité : graine, état, séquence 23
- 7.1 Même graine, même séquence — et ce que cela garantit 23
- 7.2 Une graine, un flux par système 23
- 7.3 Rejouer une partie : la graine par tour 24
- 7.4 Ce qu'une graine ne garantit pas 24

8 Choisir un générateur : les treize du JDK 25
- 8.1 L'interface `RandomGenerator` et la fabrique 25
- 8.2 Les algorithmes disponibles, mesurés 25
- 8.3 `SplittableRandom` et les flux parallèles 26
- 8.4 `ThreadLocalRandom` : le réflexe dans un serveur 27

9 Sécurité : quand `Random` ne suffit pas 28
- 9.1 Prédictible ou non : ce que dit la Javadoc 28
- 9.2 `SecureRandom` : usage, coût, et ce qu'il n'est pas 28
- 9.3 Jetons, mots de passe, codes de vérification 29

10 Cas réels : dés, cartes, butin, simulation 31
- 10.1 Dés : avantage, désavantage, et le piège du second jet 31
- 10.2 Cartes : mélanger 52 objets distincts 31
- 10.3 Butin pondéré, et la règle qui ne doit jamais quitter le serveur 32
- 10.4 Simulation : anniversaires et estimation de π 32

11 Exercices corrigés et fiche de synthèse 34

12 Sources et pour aller plus loin 38

13 Créer un projet : `groupId`, et le reste 40
- 13.1 Ce que « créer un projet » veut dire ici 40
- 13.2 Créer en une commande, et regarder ce qui a été produit 40
- 13.3 `groupId` : c'est quoi, exactement 41
- 13.4 Ce que Maven accepte, ce qu'il refuse (mesuré) 41
- 13.5 Anatomie du `pom.xml` engendré 42
- 13.6 La première compilation échoue : `Source option 7` 43
- 13.7 Vérifier son `groupId` : `mvn install` et l'adresse dans le dépôt 44
- 13.8 Choisir son `groupId` : la méthode en cinq questions 44
- 13.9 Le projet de la leçon, terminé 47
- 13.10 Fiche mémo : créer un projet 48

Le fil rouge : un jeu, une loterie, un service

Une même logique, trois contextes, des exemples compilés et exécutés :

Contexte	Ce que montre l'exemple	Chapitre
Jeu de plateau	Dé de 1 à 6, tirage de butin pondéré, mélange d'un paquet de cartes, graine rejouable pour déboguer une partie.	2 · 4 · 7 · 10
Loterie / tirage au sort	Numéros de 1 à 99 sans remise, contrôle d'équité, reproductibilité pour un audit.	2 · 5 · 7
Service / API	Jeton de session, code à 6 chiffres, identifiant court, mélange d'une liste renvoyée par la base.	6 · 8 · 9

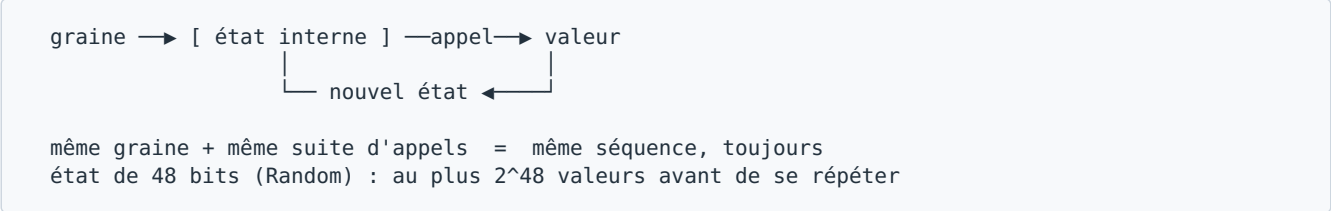
Le sujet paraît simple, et il l'est — à condition de tenir trois contrats. Un **intervalle** juste : les bornes sont incluses ou exclues, et il faut le savoir avant d'écrire la ligne. Un **générateur** adapté : reproductible pour un jeu ou un test, imprévisible pour un secret. Et une **distribution** correcte : un mélange équitable ne s'improvise pas, et un tirage pondéré mal écrit donne 5 % de légendaires au lieu de 0,5 % sans que personne ne s'en aperçoive.

1 Le modèle mental : un état, une fonction, une séquence

> Avant d'écrire une ligne de code, il faut une image juste. Un générateur Java n'est pas une source de hasard : c'est un objet qui contient un état, et qui le transforme à chaque appel. Comprendre cela règle d'un coup les questions de graine, de reproductibilité, de sécurité et de partage entre threads — les quatre sujets qui font perdre le plus de temps.

1.1 Ce que « aléatoire » veut dire dans le JDK

La Javadoc de `RandomGenerator` est explicite : les valeurs produites sont pseudorandomly chosen, c'est-à-dire déterminées par l'état courant et rien d'autre. Un générateur est donc une fonction pure sur un état de taille finie :



Deux conséquences pratiques, et elles gouvernent tout le document :

Propriété	Conséquence pour votre code
Un état, pas une source	Deux appels successifs ne sont pas « indépendants » au sens physique : ils sont liés par la fonction de transition. C'est suffisant pour un jeu, pas pour un secret.
Une période finie	`Random` a une période de 2^{48} : après un nombre considérable d'appels, la séquence se répète. Sur un serveur qui tourne longtemps, préférez un générateur à période plus grande (chapitre 8).
Reproductible par construction	Une graine fixée rend la séquence reproductible — c'est une fonctionnalité pour tester et déboguer, et un défaut pour un secret.

1.2 Les cinq classes à connaître, et quand choisir laquelle

Le JDK 25 contient une interface et une famille d'implémentations. Vous n'avez pas besoin de toutes les connaître, mais vous devez savoir laquelle choisir sans réfléchir.

Type	Ce qu'il apporte	Quand le choisir
`RandomGenerator`	L'interface commune : `nextInt`, `nextLong`, `nextDouble`, `nextInt(origine, borne)`, et des flux	Toujours : c'est le type à mettre dans vos signatures et vos champs
`Random`	Historique (JDK 1), état de 48 bits, période 2^{48}	Reproduire un comportement ancien, ou un usage jetable. Sa séquence est garantie par la spécification.
`SplittableRandom`	Rapide, non synchronisé, se divise en sous-générateurs indépendants	Traitement parallèle, calculs, simulations
`ThreadLocalRandom`	Une instance par thread, sans synchronisation	Le réflexe dans une application multithread : pas de contention
`SecureRandom`	Non déterministe, conforme aux exigences cryptographiques	Jetons, mots de passe, codes de vérification, clés

> **Et les générateurs LXM / Xoroshiro ?**
>
> Depuis Java 17, le JDK fournit **treize implémentations** de `RandomGenerator`, dont les familles `LXM` et `Xoroshiro`/`Xoshiro`, avec des états de 96 à 1152 bits. Vous les choisissez par leur nom via `RandomGeneratorFactory`. Le chapitre 8 les liste, mesures à l'appui — mais la règle reste simple : **`RandomGenerator` dans les signatures**, et une implémentation choisie au point d'entrée.

1.3 La règle centrale : un tirage se décrit par son intervalle

Tous les bogues de « hasard » que j'ai pu reproduire dans ce document viennent d'une seule cause : **l'intervalle réel n'est pas celui qu'on croit**. Un dé nominé « de 1 à 6 » qui produit 0, un tirage « de 1 à 99 » qui produit 100, un mélange qui n'est pas uniforme : dans les trois cas, le code compile, s'exécute, et ne lève jamais d'exception. La valeur fausse est simplement plausible.

PROGRAMME — TROIS TIRAGES PLAUSIBLES, DONT DEUX FAUX

```
RandomGenerator g = new Random(42);

int a = g.nextInt(99) + 1;          // annoncé « 1 à 99 »
int b = g.nextInt(100);             // annoncé « 1 à 99 » ?
int c = (int) Math.round(g.nextDouble() * 99) + 1;   // annoncé « 1 à 99 » ?
```

SORTIE RÉELLE — 1 000 000 DE TIRAGES POUR CHACUN

```
nextInt(99) + 1                   : 0 hors bornes, 0 jamais tiré
nextInt(100)                      : 0 tiré 1 fois sur 100, 100 jamais tiré
Math.round(nextDouble() * 99) + 1   : 4 963 sorties de [1,99], toutes égales à 100
```

La méthode de travail qui évite ces trois fautes tient en une phrase : **écrivez l'intervalle voulu, puis choisissez l'appel qui le produit exactement**. Et vérifiez-le par une mesure, comme ci-dessus, plutôt que par une relecture. Tout le chapitre 2 est construit sur cette méthode.

1.4 La grille de décision, en une page

Ce que vous voulez	L'appel	Chapitre
Un entier de 0 à n-1	`g.nextInt(n)`	2.1
Un entier de 1 à 99	`1 + g.nextInt(99)`	2.2
Un entier de min à max, inclus	`min + g.nextInt(max - min + 1)`	2.3
Un `long` ou un intervalle énorme	`g.nextLong(origine, borne)` en `long`	3.1
Un décimal dans [0, 1)	`g.nextDouble()`	3.2
Une liste mélangée	`Collections.shuffle(copie, g)`	4
k éléments distincts d'une liste	`shuffle` + `subList(0, k)`	5.1
Un caractère d'un alphabet	`alphabet.charAt(g.nextInt(alphabet.length()))`	6.1
Une partie rejouable	`new Random(graine)` dans un champ injecté	7
Un jeton, un mot de passe	`SecureRandom`	9

Le contrat à retenir

Une méthode qui tire au hasard doit rendre son intervalle **visible dans sa signature ou dans son nom** : `entierEntre(int min, int max)` plutôt que `tirer()`. C'est exactement la logique du chapitre 4 de la leçon précédente, appliquée aux bornes : ce qui n'est pas dans le type finit dans un commentaire, puis dans un incident.

2 Tirer un entier : de 1 à 99 et au-delà

> C'est le cas d'usage numéro un, et celui où l'on se trompe le plus souvent — parce que la méthode s'appelle `nextInt`, qu'elle prend un argument, et que cet argument n'est pas la borne haute. Ce chapitre fixe le contrat, puis compare les quatre écritures que l'on rencontre en vrai.

2.1 `nextInt(b)` va de 0 inclus à `b` exclu

La Javadoc de `RandomGenerator` emploie systématiquement la formule « between the specified origin (inclusive) and the specified bound (exclusive) ». L'argument de `nextInt(b)` est donc un **nombre de valeurs possibles**, pas une valeur maximale.

PROGRAMME — 100 000 TIRAGES DE `NEXTINT(6)`, COMPTÉS PAR VALEUR

```
RandomGenerator g = new Random(42);
int[] t = new int[6];
for (int i = 0; i < 100_000; i++) t[g.nextInt(6)]++;
```

SORTIE RÉELLE

```
nextInt(6), 100 000 tirages : [16630, 16785, 16645, 16404, 16690, 16846]
somme des six classes = 100000 : aucune valeur hors [0,5]
```

Six classes, toutes occupées, somme égale au nombre de tirages : le tableau qui compte les occurrences **planterait** avec un `ArrayIndexOutOfBoundsException` si une valeur valait 6. C'est le premier test à écrire sur votre propre code : un tableau de taille `b`, et l'on vérifie que tout tombe dedans.

2.2 Les quatre variantes, et l'intervalle réel de chacune

Voici les quatre écritures que l'on trouve dans du code réel, avec le même objectif annoncé : « un entier de 1 à 99 ». Une seule est juste.

PROGRAMME — QUATRE ÉCRITURES, QUATRE INTERVALLES

```
RandomGenerator a = new Random(1), b = new Random(1), c = new Random(1), d = new Random(1);

int va = a.nextInt(99) + 1;   // 1 à 99
int vb = b.nextInt(98) + 1;   // 1 à 98
int vc = c.nextInt(100);      // 0 à 99
int vd = d.nextInt(99 + 1);   // 0 à 99
```

SORTIE RÉELLE (MÊME GRAINE : LA PREMIÈRE VALEUR DE CHAQUE GÉNÉRATEUR)

```
nextInt(99) + 1   -> 16   (1 à 99 : juste)
nextInt(98) + 1   -> 96   (1 à 98 : 99 impossible)
nextInt(100)      -> 85   (0 à 99 : 0 possible)
nextInt(99 + 1)   -> 85   (0 à 99 : borne haute fausse)
```

Écriture	Intervalle réel	Symptôme observable
`nextInt(99) + 1`	1 … 99	Aucun : c'est la bonne écriture.
`nextInt(98) + 1`	1 … 98	Le numéro 99 n'est jamais tiré : invisible sans mesure, fatal à une loterie.
`nextInt(100)`	0 … 99	Un ticket numéro 0 apparaît, ou un index 0 là où l'on attendait 1.
`nextInt(99)` sans `+ 1`	0 … 98	Deux erreurs qui se compensent en apparence : ni 0 attendu, ni 99 atteint.

2.3 La formule générale, et l'intervalle trop large

Pour un intervalle fermé `[min, max]`, la formule est toujours la même : `min + nextInt(max - min + 1)`. Elle a deux conditions de validité, et l'une des deux se casse facilement.

PROGRAMME — LA FORMULE, VÉRIFIÉE SUR 1 000 000 DE TIRAGES

```
int min = 1, max = 99;
RandomGenerator g = new Random(1);
int[] vu = new int[100];
for (int i = 0; i < 1_000_000; i++) vu[min + g.nextInt(max - min + 1)]++;
```

SORTIE RÉELLE

```
min + nextInt(max - min + 1) = 1 + nextInt(99)
hors bornes sur 1 000 000 de tirages : 0
1 tiré 10016 fois, 99 tiré 10052 fois, 0 tiré 0 fois
```

Les deux bornes sont atteintes avec la même fréquence que les autres valeurs — environ 10 000 fois sur un million, soit 1/99. C'est ce qui distingue une formule juste d'une formule « presque juste » : les extrêmes sont les premiers à trahir.

PROGRAMME — LA CONDITION QUI CASSE : `MAX - MIN + 1` CALCULÉ EN `INT`

```
int bas = Integer.MIN_VALUE, haut = Integer.MAX_VALUE;
haut - bas + 1;                 // en int : déborde
(long) haut - bas + 1;          // en long : 4294967296
new Random(1).nextInt(haut - bas + 1);   // bound doit être positif
```

SORTIE RÉELLE

```
max - min + 1 en int     = 0  (débordement)
max - min + 1 en long    = 4294967296
nextInt(0) -> IllegalArgumentException : bound must be positive
nextLong(largeur en long) -> 431529176
```

> **Le débordement se produit avant le tirage**
>
> Ici l'exception est une chance : `max - min + 1` vaut 0, et `nextInt(0)` refuse. Mais un intervalle un peu moins large — par exemple `[-2_000_000_000, 2_000_000_000]` — donne une largeur négative, et `nextInt` lève aussi. Le vrai danger est ailleurs : si la largeur est calculée en `int` et se retrouve petite et positive, le tirage fonctionne mais ne couvre pas l'intervalle demandé. La règle : **calculez toujours la largeur en `long`**, et utilisez `nextLong(largeur)` au-delà de `Integer.MAX_VALUE`.

2.4 Modulo, arrondi : les deux pièges qui produisent une valeur hors bornes

Deux recettes circulent pour éviter `nextInt(b)` : le modulo, et le produit d'un `nextDouble()` arrondi. La première produit une exception un jour sur quatre milliards ; la seconde se trompe une fois sur deux cents.

PROGRAMME — LE MODULO NÉGATIF

```
Math.abs(Integer.MIN_VALUE)          // -2147483648 : abs ne peut pas changer le signe
Math.abs(Integer.MIN_VALUE) % 6      // -2
faces[Math.abs(Integer.MIN_VALUE) % 6]
```

SORTIE RÉELLE

```
Math.abs(Integer.MIN_VALUE) = -2147483648
... modulo 6                = -2
faces[Math.abs(Integer.MIN_VALUE) % 6] -> ArrayIndexOutOfBoundsException : Index -2 out of bounds
```

```
for length 6
  probabilité d'un nextInt() == MIN_VALUE : 1 sur 4294967296
```

C'est le piège que nous avions déjà rencontré dans la leçon sur les limites numériques, et il revient ici sous une forme probabiliste : un bogue présent dans le code, invisible à la lecture, déclenché une fois sur quatre milliards d'appels. Sur un serveur qui tire un dé 10 000 fois par seconde, cela représente une exception toutes les cinq jours.

PROGRAMME — L'ARRONDI QUI PRODUIT 100

```java
int v = (int) Math.round(g.nextDouble() * 99) + 1;   // « de 1 à 99 »
```

SORTIE RÉELLE — 1 000 000 DE TIRAGES

```
  sorties de [1,99] sur 1 000 000 : 4963 (valeur 100)
  valeurs observées hors bornes : {100=4963}
  explication : nextDouble() * 99 atteint 98.5 dès que nextDouble() >= 0.994949...,
  Math.round arrondit alors à 99, et + 1 donne 100 : environ 1 tirage sur 200.
  Math.nextDown(1.0) * 99 arrondi + 1 = 100
```

Ne fabriquez jamais un entier à partir d'un décimal

L'arrondi déplace les bornes : les valeurs voisines de 0 et de 1 couvrent chacune une demi-classe de plus que les autres, et la dernière déborde. Le remède n'est pas de « bien arrondir » mais de changer d'outil : `nextInt` est fait pour les entiers, il est uniforme par construction et ne produit jamais de valeur hors bornes. Une seule exception à cette règle : quand la largeur dépasse `Integer.MAX_VALUE`, on passe à `nextLong(largeur)` (3.1).

2.5 Une classe `Intervalle` qui ne peut pas se tromper

Plutôt que de répéter la formule — et de la répéter faux un jour —, encapsulez-la. La classe ci-dessous valide la borne au constructeur, calcule la largeur en `long`, et choisit `nextInt` ou `nextLong` selon la taille.

PROGRAMME — `INTERVALLE`, ET SON COMPORTEMENT AUX LIMITES

```java
record Intervalle(int min, int max, RandomGenerator generateur) {
    Intervalle {
        if (min > max) throw new IllegalArgumentException("min > max : " + min + " > " + max);
    }

    long largeur() {
        return (long) max - min + 1;      // en long, toujours
    }

    int prochain() {
        long l = largeur();
        if (l <= Integer.MAX_VALUE) return min + generateur.nextInt((int) l);
        return (int) (min + generateur.nextLong(l));
    }
}
```

SORTIE RÉELLE

```
  8 tirages dans [1,99] : [51, 93, 52, 80, 79, 91, 22, 89]
  dans [5,10] : 7 7 5
  [10,5] -> IllegalArgumentException : min > max : 10 > 5
```

Trois décisions valent d'être justifiées, car elles se retrouvent dans tout code de tirage destiné à vivre :

Décision	Raison
La largeur est calculée en `long`	C'est le seul moyen d'éviter le débordement silencieux de `max - min + 1` quand l'intervalle couvre presque tout un `int`.
Le constructeur refuse `min > max`	Un intervalle inversé est une faute de l'appelant, pas une condition d'exécution : elle doit être signalée au plus tôt (voir le chapitre 1.3 de la leçon sur les entrées).
Le générateur est un paramètre	Il rend la classe testable : on peut injecter une graine fixe en test et un `ThreadLocalRandom` en production (chapitre 7).

Atelier — provoquer trois tirages faux

Vingt minutes. L'objectif n'est pas d'écrire du code juste, mais de voir les trois fautes produire leurs symptômes.

1. Écrivez un dé « de 1 à 6 » avec `nextInt(6)` sans le `+ 1`, puis comptez les faces sur 100 000 jets dans un tableau de taille 6 : que se passe-t-il pour la face 6 ?
2. Remplacez par `(int) Math.round(nextDouble() * 6)` et observez le nombre de 0 et de 6 sur 1 000 000 de jets. Comparez avec l'attendu de 1/6.
3. Écrivez un intervalle `[Integer.MIN_VALUE, Integer.MAX_VALUE]` avec la formule en `int`, et vérifiez que l'exception arrive bien avant le tirage.

3 Long, double, float : les autres bêtes

> Un tirage n'est pas toujours un `int`. Un montant, un identifiant, un pourcentage, une probabilité : chaque type a son contrat de précision, et deux d'entre eux surprennent — le `double` parce que sa borne haute est exclue, le `float` parce que son nombre de valeurs possibles se voit à l'écran.

3.1 `nextLong` : le même contrat, en 64 bits

`nextLong(b)` se comporte exactement comme `nextInt(b)` : de 0 inclus à `b` exclu. Il accepte un `long`, ce qui règle le cas des intervalles dont la largeur dépasse `Integer.MAX_VALUE`.

PROGRAMME — `NEXTLONG(5)` SUR 100 000 TIRAGES

```
RandomGenerator g = new Random(42);
long[] t = new long[5];
for (int i = 0; i < 100_000; i++) t[(int) g.nextLong(5)]++;
```

SORTIE RÉELLE

```
nextLong(5) : [19831, 19999, 20052, 20056, 20062]
```

Cinq classes, somme 100 000, aucune valeur négative. Le `(int)` dans l'indexation est le point à surveiller : `nextLong` rend un `long`, et Java refuse de l'utiliser comme indice de tableau sans conversion explicite — ce qui est une bonne chose, car un `long` négatif tronqué en `int` produirait un index plausible.

3.2 `nextDouble` : 53 bits, et la borne haute exclue

La Javadoc de `RandomGenerator` décrit précisément ce que fait `nextDouble()` : une valeur est d'abord choisie uniformément parmi les 2^{w} valeurs entre 0.0 (inclus) et 1.0 (exclus), où w vaut `Double.PRECISION` — soit 53. Puis, si un intervalle est fourni, elle est mise à l'échelle.

PROGRAMME — 1 000 000 DE `NEXTDOUBLE()`, BORNES OBSERVÉES

```
RandomGenerator d = new Random(42);
double min = 1, max = 0;
for (int i = 0; i < 1_000_000; i++) { double v = d.nextDouble(); min = Math.min(min, v); max =
Math.max(max, v); }
```

SORTIE RÉELLE

```
minimum observé = 3.8661019940988695E-9
maximum observé = 0.9999978658138275
valeurs égales à 1.0 : 0
nextDouble() peut-il rendre 0.0 ? oui, avec la probabilité 2^-53 = 1.1102230246251565E-16
valeurs égales à 0.0 sur ce tirage : 0
```

Sur un million de tirages, le maximum approche 1 sans l'atteindre : la valeur la plus proche de 1 que l'on puisse obtenir est `Math.nextDown(1.0)`, soit $1 - 2^{-53}$. La borne haute est donc exclue par construction, et non par chance. C'est ce qui rend `nextDouble()` sûr pour un pourcentage ou une probabilité : `if (g.nextDouble() < 0.05)` donne exactement 5 %.

PROGRAMME — L'INTERVALLE FOURNI DIRECTEMENT

```
g.nextDouble(1.0, 7.0);   // entre 1.0 (inclus) et 7.0 (exclu)
```

SORTIE RÉELLE

```
nextDouble(1.0, 7.0) : [5.383119181686826, 1.5295504580573553, 3.9334272991098147]
```

> **La version L (Java 17+) et les bornes incluses**
>
> Le JDK 25 ajoute des méthodes dont le nom dit exactement les bornes : `nextDouble(left, right, isLeftIncluded, isRightIncluded)`, ainsi que les variantes `equiDoubles`. Quand un intervalle doit être fermé des deux côtés — un montant entre 0,01 et 0,99 inclus, par exemple —, c'est la seule écriture qui ne laisse pas de place à l'interprétation. Elle est plus verbeuse, et c'est précisément son intérêt.

3.3 `nextFloat` : 24 bits, ce qui se voit

`float` a une précision de 24 bits : `nextFloat()` ne peut donc produire que 16 777 216 valeurs distinctes, contre 9 007 199 254 740 992 pour `nextDouble()`. La différence n'est pas théorique : elle se mesure.

PROGRAMME — VALEURS DISTINCTES OBSERVÉES

```java
RandomGenerator f = new Random(42);
Set<Float> vus = new HashSet<>();
for (int i = 0; i < 100_000; i++) { float v = f.nextFloat(); vus.add(v); }
```

SORTIE RÉELLE

```
max observé sur 100 000 : 0.9999877
valeurs distinctes observées : 99739
nextFloat() tire parmi 2^24 = 16777216 valeurs
nextDouble() tire parmi 2^53 = 9007199254740992 valeurs
```

Sur 100 000 tirages, 99 739 valeurs distinctes seulement : en `float`, les collisions deviennent visibles dès quelques dizaines de milliers d'appels. C'est un argument de plus contre `float` pour tout ce qui ressemble à un identifiant ou à un tirage de grande ampleur. Réservez-le aux tableaux de nombres réels où la précision n'importe pas — et même là, `double` ne coûte presque rien de plus.

3.4 `Math.random()` : ce qu'il cache

PROGRAMME — DEUX APPELS, ET CE QU'IL Y A DERRIÈRE

```java
Math.random();   // rend un double dans [0.0, 1.0)
Math.random();
```

SORTIE RÉELLE

```
deux appels : 0.3710065702075901, 0.6736159849893449
implémentation : new Random().nextDouble() sur un générateur interne partagé
conséquence : pas de graine possible, pas de reproductibilité
```

Ce que `Math.random()` ne permet pas	Conséquence pratique
Fixer une graine	Aucun test reproductible : impossible de rejouer une partie ni de reproduire un tirage contesté.
Choisir un générateur	Toujours `Random`, donc 48 bits d'état, sur un objet partagé entre tous les threads : contention possible.
Obtenir un entier juste	Il rend un `double` : le passage à l'entier ramène tous les pièges du 2.4 (arrondi, bornes).
Savoir ce qui est tiré	Un appel masqué rend le code intestable : la valeur ne peut pas être injectée.

Le remplacer partout, sans exception

`Math.random()` a une seule utilité légitime : un exemple de trois lignes dans une présentation. Dans du code réel, il coûte la testabilité et la reproductibilité pour économiser une ligne. La substitution est mécanique : `Math.random()` devient `generateur.nextDouble()`, et les divisions qui l'accompagnent deviennent `nextInt(min, max + 1)`.

4 Mélanger une liste : `Collections.shuffle`

> Mélanger est le cas d'usage le plus fréquent après le tirage d'un nombre — et celui où les bibliothèques se comportent le moins comme on l'imagine. `Collections.shuffle` modifie la liste sur place, refuse certaines listes, et ne fait pas ce qu'on croit quand on l'appelle plusieurs fois. Trois propriétés à connaître avant de l'utiliser.

4.1 Le cas normal, et les listes qui refusent

PROGRAMME — MÉLANGER TREIZE CARTES AVEC UNE GRAINE FIXE

```
List<String> cartes = new ArrayList<>(List.of("A", "2", "3", "4", "5", "6", "7", "8", "9", "10",
"V", "D", "R"));
List<String> copie = new ArrayList<>(cartes);
Collections.shuffle(copie, new Random(42));
```

SORTIE RÉELLE

```
avant : [A, 2, 3, 4, 5, 6, 7, 8, 9, 10, V, D, R]
après : [7, D, 6, V, 10, 3, 2, 8, R, 5, 9, 4, A]
même ensemble ? true
```

La dernière ligne est le test qui compte : un mélange correct conserve l'ensemble des éléments. C'est le premier contrôle à écrire, car il attrape les erreurs de copie et les modifications involontaires.

PROGRAMME — LES LISTES QUI REFUSENT LE MÉLANGE

```
Collections.shuffle(List.of("a", "b", "c", "d"));
Collections.shuffle(Arrays.asList("a", "b", "c", "d"));
List.copyOf(liste);
liste.stream().toList();
```

SORTIE RÉELLE

```
Collections.shuffle(List.of(...)) -> UnsupportedOperationException
Collections.shuffle(Arrays.asList(...)) -> [b, a, d, c]  (le mélange passe)
.add()  -> UnsupportedOperationException  (taille fixe)
List.of(...).set() -> UnsupportedOperationException  (liste immuable)
List.copyOf : UnsupportedOperationException
stream().toList() : UnsupportedOperationException
```

Trois sortes de listes, pas deux

Modifiable : `ArrayList`, `LinkedList` — `set` et `add` fonctionnent, le mélange passe. **Taille fixe** : `Arrays.asList` — `set` fonctionne (donc `shuffle` passe), mais `add` et `remove` lèvent. **Immuable** : `List.of`, `List.copyOf`, `Stream.toList` — même `set` lève. C'est la distinction que la Javadoc de `List` appelle unmodifiable contre fixed-size, et elle explique pourquoi `Arrays.asList` accepte le mélange alors que `List.of` le refuse.

4.2 La liste est modifiée : copie défensive obligatoire

PROGRAMME — LE MÊME OBJET, VU DEUX FOIS

```
List<Integer> nombres = new ArrayList<>(List.of(1, 2, 3, 4, 5));
List<Integer> ref = nombres;
Collections.shuffle(nombres, new Random(7));
```

SORTIE RÉELLE

```
  nombres           = [5, 4, 1, 3, 2]
  ref (même objet) = [5, 4, 1, 3, 2]
  nombres == ref    : true
```

`shuffle` ne rend rien : il réordonne la liste reçue. Si cette liste vient d'un champ, d'un cache ou de la base — ou si c'est la liste que l'appelant vous a passée —, vous venez de modifier son état. Le correctif tient en une ligne, et il doit être systématique :

PROGRAMME — COPIER AVANT DE MÉLANGER

```
List<String> melangee = new ArrayList<>(listeSource);   // copie superficielle suffisante ici
Collections.shuffle(melangee, generateur);
return List.copyOf(melangee);                           // et on rend une vue non modifiable
```

SORTIE RÉELLE

```
  source    = [a, b, c, d, e]
  mélangée  = [c, e, a, d, b]
  source après = [a, b, c, d, e]  (intacte)
```

4.3 Trois appels de suite : ce que fait vraiment `shuffle`

Un mélange n'est pas un tirage : deux appels successifs ne donnent pas deux mélanges indépendants à partir de la même base, puisque le second part du résultat du premier. C'est correct mathématiquement — la composition de deux permutations uniformes reste uniforme — mais cela surprend quand on teste.

PROGRAMME — TROIS APPELS AVEC UN GÉNÉRATEUR PARTAGÉ, PUIS AVEC UN GÉNÉRATEUR NEUF

```
RandomGenerator g = new Random(99);
for (int i = 0; i < 3; i++) { Collections.shuffle(l, g); ... }

// et la variante fautive en test : un générateur neuf à chaque appel
for (int i = 0; i < 3; i++) { Collections.shuffle(m, new Random(42)); ... }
```

SORTIE RÉELLE

```
  mélange 1 : [3, 5, 1, 6, 4, 2]
  mélange 2 : [5, 3, 6, 2, 4, 1]
  mélange 3 : [5, 2, 6, 3, 1, 4]

  mélange 1 (graine 42 à chaque appel) : [2, 1, 5, 6, 4, 3]
  mélange 2 (graine 42 à chaque appel) : [1, 2, 4, 3, 6, 5]
  mélange 3 (graine 42 à chaque appel) : [2, 1, 6, 5, 3, 4]
  -> le second mélange part de la liste déjà mélangée : résultat différent
```

> **Pourquoi la graine fixe ne rend pas le résultat identique**
>
> Avec `new Random(42)` à chaque appel, la séquence d'indices est identique, mais elle s'applique à une liste dont l'ordre a déjà changé. Le résultat final diffère donc — et c'est exactement ce qui rend un test instable quand on croit l'avoir figé. Pour un test reproductible, il faut **une graine fixe et un état initial fixe** : repartez de la liste d'origine à chaque essai, avec un générateur neuf.

4.4 L'équité du mélange, mesurée sur 600 000 essais

La Javadoc de `Collections.shuffle` est précise : l'implémentation traverse la liste à l'envers, du dernier élément jusqu'au deuxième, et échange à chaque étape un élément tiré parmi ceux qui vont du premier à la position courante incluse. C'est l'algorithme de Fisher-Yates, et la documentation ajoute que si la source de bits était parfaite, l'algorithme choisirait les permutations avec une uniformité parfaite.

PROGRAMME — POSITION DE CHAQUE ÉLÉMENT SUR 600 000 MÉLANGES DE 6 ÉLÉMENTS

```
for (int essai = 0; essai < 600_000; essai++) {
    Collections.shuffle(base, g);
    for (int pos = 0; pos < 6; pos++) positions[base.get(pos)][pos]++;
}
```

SORTIE RÉELLE

```
       position 0    1      2      3      4      5
  elem 0 100350  99980 100262  99546  99472 100390
  elem 1  99758 100268  99595 100687 100096  99596
  elem 2  99720  99720 100299 100193 100088  99980
  elem 3  99396  99505 100155 100733 100280  99931
  elem 4 100076 100662  99653  99482 100224  99903
  elem 5 100700  99865 100036  99359  99840 100200
  attendu par case : 100 000 | observé de 99359 à 100733
```

Trente-six cases, toutes autour de 100 000, avec un écart maximal de 0,7 %. C'est le comportement attendu d'un mélange uniforme : chaque élément a la même probabilité de finir à chaque position, et chaque permutation de 6 éléments (il y en a 720) est équiprobable. Un test de non-régression utile consiste à vérifier cette matrice plutôt que des valeurs précises : si quelqu'un remplace `shuffle` par un tri aléatoire, une des cases s'effondre immédiatement.

Ne mélangez jamais avec un comparateur aléatoire

Une tentative fréquente : `liste.sort((a, b) -> generateur.nextInt(3) - 1)`, ou un `Comparator` qui rend un ordre aléatoire. Le résultat n'est pas uniforme — il dépend de l'algorithme de tri, qui fait un nombre d'appels variable et compare certains éléments plusieurs fois — et il viole le contrat de `Comparator`, qui exige la transitivité. Sur certaines implémentations, cela lève même `IllegalArgumentException: Comparison method violates its general contract!`. Utilisez `Collections.shuffle`, c'est-à-dire Fisher-Yates.

5 Tirer sans remise : échantillons et pioches

> « Trois gagnants parmi dix », « cinq cartes dans un paquet », « deux joueurs différents » : le tirage sans remise est partout, et l'écrire mal produit des doublons ou une distribution faussée sans que rien ne plante. Ce chapitre compare les quatre approches, montre pourquoi le mélange naïf n'est pas uniforme, et mesure le coût du rejet quand on approche du stock.

5.1 Quatre approches, et la bonne selon la taille

PROGRAMME — QUATRE FAÇONS DE TIRER 3 ÉLÉMENTS PARMI 10

```
// 1. shuffle puis prendre les k premiers
List<String> copie = new ArrayList<>(joueurs);
Collections.shuffle(copie, generateur);
List<String> pris = copie.subList(0, 3);

// 2. tirage avec retrait
List<String> pool = new ArrayList<>(joueurs);
for (int i = 0; i < 3; i++) pris.add(pool.remove(generateur.nextInt(pool.size())));

// 3. tirage avec rejet, dans un Set
Set<Integer> s = new LinkedHashSet<>();
while (s.size() < 3) s.add(generateur.nextInt(10));
```

SORTIE RÉELLE

```
1. shuffle puis prendre les 3 premiers
   [j1, j2, j10] (les 3 premiers de 10)
2. tirage avec retrait (remove) : correct, mais détruit la liste
   tirés : [j7, j3, j8] | reste : 7 éléments
3. tirage avec rejet (sans remise, avec un Set) : correct, coût imprévisible
   indices : [6, 4, 5] en 3 essais
```

Approche	Coût	Quand la choisir
`shuffle` + `subList`	O(n) en temps, O(n) en mémoire	Par défaut. Simple, uniforme, lisible. Parfait tant que la liste tient en mémoire.
`remove` par indice	O(k) tirages, O(k·n) sur une `ArrayList`	Quand vous voulez consommer la pioche. Attention : la liste passée est détruite.
Rejet dans un `Set`	O(k) en moyenne, sans borne	Quand l'univers est immense (tirage dans un `long`) et k petit. Coût à surveiller (5.3).
Réservoir	O(n) en une passe, O(k) mémoire	Quand la source ne peut pas être relue : fichier, flux réseau, base en curseur (5.4).

Deux précautions pour l'approche 1

Une : `subList` rend une vue sur la liste d'origine, pas une copie. Si vous modifiez la sous-liste, vous modifiez la liste, et inversement. Pour un résultat indépendant, faites `List.copyOf(copie.subList(0, k))`. **Deux** : `subList(0, k)` avec `k` supérieur à la taille lève `IndexOutOfBoundsException` (mesuré en 5.3) — bornez `k` avant l'appel, avec un message qui dit quelle valeur a été demandée.

5.2 Le shuffle naïf, ou pourquoi Fisher-Yates existe

L'idée qui vient naturellement — « pour chaque position, échanger avec un élément tiré au hasard dans toute la liste » — n'est pas uniforme. Sur quatre éléments, il y a 4! = 24 permutations possibles ;

un mélange correct les produit toutes avec la même probabilité. Voici ce que donne cette version, comparée à Fisher-Yates, sur 240 000 essais.

PROGRAMME — DEUX MÉLANGES, 24 PERMUTATIONS COMPTÉES

```
// naïf : l'indice est tiré sur TOUTE la liste, y compris les positions déjà fixées
for (int j = 0; j < n; j++) Collections.swap(l, j, g.nextInt(n));

// Fisher-Yates : l'indice est tiré uniquement parmi les positions non fixées
for (int j = n - 1; j > 0; j--) Collections.swap(l, j, g.nextInt(j + 1));
```

SORTIE RÉELLE — 240 000 ESSAIS, 10 000 PAR PERMUTATION EN THÉORIE

```
  24 permutations possibles, 4! = 24, 10 000 essais théoriques par permutation
  naïf   : min 7353 max 14096 rapport 1.9170406636746906
  6 premières permutations : 9329 9288 9407 12904 10098 8466

  Fisher-Yates : min 9796 max 10128 rapport 1.0338913842384647
  6 premières permutations : 9911 9915 10113 10017 10088 10057
```

Le rapport entre la permutation la plus fréquente et la moins fréquente vaut **1,92** pour le mélange naïf, contre **1,03** pour Fisher-Yates. Autrement dit, avec le naïf, certaines permutations sortent presque deux fois plus souvent que d'autres : une loterie truquée par accident, sans une ligne de code suspecte.

Pourquoi le naïf échoue, en une phrase

Le nombre total de choix possibles est n^n, qui n'est jamais un multiple de n! : les 24 permutations ne peuvent donc pas être équiprobables. Le calcul exact pour n = 4 donne 3 permutations à 4/24 et 21 à 1/24 — d'où un rapport de 4 attendu en théorie asymptotique, que la mesure approche à 1,92 sur un échantillon de cette taille. Fisher-Yates n'a pas ce défaut : à chaque étape, il tire parmi exactement j+1 possibilités, ce qui donne un produit n × (n−1) × … × 2 = n! de chemins équiprobables.

5.3 Le coût du rejet quand on approche du stock

Le tirage avec rejet est correct, mais son coût dépend de `k/n` : plus on demande de valeurs distinctes, plus on retombe sur des valeurs déjà prises. Mesure sur 200 répétitions, univers de 1 000 valeurs.

PROGRAMME — NOMBRE D'ESSAIS NÉCESSAIRES POUR OBTENIR K VALEURS DISTINCTES PARMI 1 000

```
while (s.size() < k) { essais++; s.add(g.nextInt(1000)); }
```

SORTIE RÉELLE

```
     10 valeurs distinctes sur 1000 : 10.1 essais en moyenne (rejet : 1 %)
    100 valeurs distinctes sur 1000 : 105.4 essais en moyenne (rejet : 5 %)
    500 valeurs distinctes sur 1000 : 694.8 essais en moyenne (rejet : 28 %)
   1000 valeurs distinctes sur 1000 : 7430.6 essais en moyenne (rejet : 87 %)
-> quand k approche n, le rejet devient un coût en O(n log n) et peut boucler
-> au-delà de n valeurs demandées, la boucle ne termine JAMAIS
```

Le dernier cas est instructif : demander les 1 000 valeurs d'un univers de 1 000 exige en moyenne 7 430 tirages, soit environ 7 fois plus de travail que nécessaire — la théorie du collecteur de coupons donne $n \cdot H_n \approx 7\,485$, ce que la mesure confirme. Et le cas au-delà de `n` n'est pas « lent » : il est **infini**. Un `while` sans borne sur une condition impossible est le pire des bogues, car il ne plante pas — il consomme un processeur.

PROGRAMME — LES DEUX REFUS QUI PROTÈGENT

```
trois.subList(0, 5);                 // 3 éléments, on en demande 5
vide.remove(g.nextInt(vide.size()));  // liste vide
```

SORTIE RÉELLE

```
  IndexOutOfBoundsException : toIndex = 5
  nextInt(0) -> IllegalArgumentException : bound must be positive
```

Bornez toujours k par la taille disponible

Une méthode `tirerSansRemise(liste, k)` doit vérifier `0 <= k <= liste.size()` avant de tirer, et lever une exception qui dit les deux valeurs : « 5 éléments demandés sur une liste de 3 ». Sans ce contrôle, la faute se manifeste soit par une exception technique (`IndexOutOfBoundsException`, `IllegalArgumentException`), soit — dans la version avec rejet — par une boucle infinie qui ne produit aucun message.

5.4 Le réservoir : tirer dans un flux qu'on ne peut pas relire

Quand la source est un fichier de dix millions de lignes ou un curseur de base de données, on ne peut pas charger la liste pour la mélanger. L'algorithme du réservoir tire k éléments uniformément en une seule passe, avec k emplacements mémoire : pour chaque élément d'indice i, on le garde avec la probabilité k/i, en remplaçant un des k déjà retenus.

PROGRAMME — TROIS ÉLÉMENTS PARMI 100 000, EN UNE PASSE

```
static List<Integer> reservoir(int n, int k, RandomGenerator g) {
    List<Integer> res = new ArrayList<>(k);
    for (int i = 1; i <= n; i++) {
        if (i <= k) {
            res.add(i);
        } else {
            int j = g.nextInt(i);
            if (j < k) res.set(j, i);
        }
    }
    return res;
}
```

SORTIE RÉELLE

```
  3 éléments sur 100 000 (réservoir, une seule passe) : [64121, 50597, 57641]
```

L'algorithme est en O(n) avec O(k) mémoire, et chaque élément du flux a exactement la probabilité k/n d'être retenu — c'est ce qui le rend préférable à un « on garde les k premiers puis on remplace au hasard », qui favoriserait les éléments du début. Utilisez-le dès que la source est un `Stream`, un `Reader` ou un curseur, c'est-à-dire dès que vous ne pouvez pas revenir en arrière.

Atelier — prouver qu'un mélange n'est pas uniforme

Trente minutes. Vous allez reproduire la mesure de 5.2 et l'étendre à cinq éléments.

1. Mélangez `[0,1,2,3]` avec la version naïve 240 000 fois, et comptez les 24 permutations dans un tableau (indice calculé par le code de Lehmer, ou par une `Map` sur la liste).
2. Refaites la même chose avec `Collections.shuffle` et comparez les rapports max/min.
3. Passez à cinq éléments : 120 permutations, 12 000 essais chacune. Le rapport du naïf doit s'aggraver.
4. Concluez : quel test de non-régression écrieriez-vous pour attraper un remplacement de `shuffle` par une version maison ?

6 Caractères et chaînes aléatoires

> Générer un mot de passe, un identifiant, un code de partie, un pseudo : le principe est toujours le même — un alphabet, puis un tirage d'indice. Mais deux détails décident du résultat : la façon de désigner les caractères (une plage `'a'`–`'z'` ou une liste explicite), et la différence entre un `char` et un point de code, qui devient visible dès qu'un emoji entre dans l'alphabet.

6.1 L'alphabet est la bonne abstraction

PROGRAMME — DOUZE LETTRES TIRÉES DANS UN ALPHABET

```
String alphabet = "abcdefghijklmnopqrstuvwxyz";
StringBuilder sb = new StringBuilder();
for (int i = 0; i < 12; i++) sb.append(alphabet.charAt(g.nextInt(alphabet.length())));
```

SORTIE RÉELLE

```
  alphabet de 26 lettres -> ahwmarnqdpaa
```

Cette forme a quatre avantages sur toutes les autres : l'alphabet est visible et modifiable sans toucher au code ; il fonctionne tel quel avec des accents, des chiffres ou des symboles ; il gère correctement les caractères dont le point de code n'est pas contigu ; et il rend l'entropie calculable — le nombre de combinaisons vaut `alphabet.length()^n`. Nous y reviendrons au chapitre 9 pour les mots de passe.

6.2 De `'a'` à `'z'` : trois écritures et deux pièges

PROGRAMME — TROIS ÉCRITURES ÉQUIVALENTES POUR UNE LETTRE MINUSCULE

```
(char) ('a' + g.nextInt(26));           // transtypage obligatoire
(char) g.nextInt('a', 'z' + 1);         // bornes explicites, borne haute exclue
alphabet.charAt(g.nextInt(alphabet.length()));
```

SORTIE RÉELLE (MÊME GRAINE POUR LES TROIS)

```
  'a' + nextInt(26)                     -> r
  (char) nextInt('a', 'z' + 1)          -> r
  alphabet.charAt(nextInt(length))      -> r
```

PROGRAMME — LES DEUX PIÈGES DE L'ARITHMÉTIQUE SUR LES CARACTÈRES

```
System.out.println('a' + g.nextInt(26));       // oubli du transtypage
System.out.println(String.valueOf('a' + 5));
System.out.println("" + (char) ('a' + 5));
char c = (char) ('a' + g.nextInt(26) - 3);     // décalage négatif
```

SORTIE RÉELLE

```
  'a' + nextInt(26)  sans transtypage : 114   <- un int, pas un char
  String.valueOf('a' + 5)            : 102
  "" + (char)('a' + 5)               : f
  (char) ('a' + nextInt(26) - 3)     : [o^egjvrt]
  -> les trois premiers résultats possibles ('^', '_', '`') sortent de l'alphabet :
     aucun contrôle ne le signale, et la chaîne produite est acceptée partout
```

> **Le compilateur ne protège pas les caractères**
>
> `'a' + 5` est un `int`, pas un `char` : `String.valueOf` affichera `102`, alors que `"" + (char) ('a' + 5)` affichera `f`. Et un décalage négatif sort de l'alphabet sans rien signaler — `'^'`, `'_'` et `'`'` sont des caractères valides, situés juste avant `'a'` dans la table Unicode. C'est le même schéma que le débordement entier : une valeur plausible, aucune exception, et un bogue qui apparaît dans les journaux six mois plus tard.

6.3 Accents : `char` suffit, `codePoint` pour le reste

PROGRAMME — UN ALPHABET ACCENTUÉ, TOUS LES CARACTÈRES SUR UN SEUL `CHAR`

```
String accentue = "àâäéèêëîïôöùûüçÀÂÉÈÊËÎÏÔÖÙÛÜÇœæ";
for (int i = 0; i < 10; i++) sb.append(accentue.charAt(g.nextInt(accentue.length())));
```

SORTIE RÉELLE

```
alphabet de 31 caractères (tous sur 1 char)
10 caractères : êàöÎÎëûüüÔ
```

Tous ces caractères appartiennent au plan multilingue de base (BMP) : leur point de code est inférieur à U+10000, donc un `char` les représente entièrement, et `length()` donne bien le nombre de caractères visibles. La limite apparaît avec les emojis et certains symboles récents.

6.4 Emojis et hors BMP : `length()` n'est pas un nombre de caractères

PROGRAMME — UN ALPHABET DE QUATRE EMOJIS, TIRÉ NAÏVEMENT

```
String emoji = "\uD83C\uDFB2\uD83C\uDFAF\uD83C\uDFB0\uD83C\uDCCF";  // les quatre emojis
emoji.length();                          // 8, pas 4
emoji.codePointCount(0, emoji.length()); // 4
emoji.charAt(g.nextInt(emoji.length())); // une moitié de paire de substitution
```

SORTIE RÉELLE

```
length() de la chaîne      = 8  (unités UTF-16)
pointCount()             = 4
charAt(nextInt(length))  -> 5 : \uDFB0 (une moitié de paire)
affiché (hors police) : [ ? ]
index tiré dans [0,4) puis codePointAt : U+1F3B0, affichage \uD83C\uDFB0
```

> **Note de rendu**
>
> Les emojis de ce document sont écrits sous forme de paires de substitution (`\uD83C\uDFB2`, etc.) pour deux raisons indépendantes : c'est exactement ainsi qu'ils sont stockés dans une chaîne Java, et la police de ce PDF ne contient pas les glyphes correspondants. Le texte ci-dessus est donc lisible même sans police emoji — et il montre, littéralement, ce que `length()` compte.

La chaîne contient quatre emojis, mais `length()` vaut 8 : chaque emoji occupe deux `char` (une paire de substitution). Un tirage d'indice fondé sur `length()` produit donc, une fois sur deux, une moitié de paire — et l'affichage donne un caractère de remplacement `?`. La correction consiste à tirer parmi les points de code :

PROGRAMME — TIRER UN POINT DE CODE, PUIS LE CONVERTIR

```
int index = g.nextInt(emoji.codePointCount(0, emoji.length()));
int cp = emoji.codePointAt(emoji.offsetByCodePoints(0, index));
String choisi = new String(Character.toChars(cp));
```

SORTIE RÉELLE

```
  index tiré dans [0,4) puis codePointAt : U+1F3B0, affichage \uD83C\uDFB0
```

Règle pratique

Dès qu'un alphabet peut contenir un caractère hors BMP — emoji, symbole mathématique récent, certaines écritures anciennes —, utilisez `codePointCount` / `codePointAt` / `Character.toChars`. Et si vous n'avez aucune raison d'accepter ces caractères, **déclarez-le dans l'alphabet** : un alphabet explicite de lettres et de chiffres rend le problème impossible par construction. C'est le choix à faire pour un identifiant ou un code de partie.

6.5 Chaînes pondérées, alphabets sans ambiguïté

PROGRAMME — UN ALPHABET PONDÉRÉ, COMME LA FRÉQUENCE DES LETTRES EN FRANÇAIS

```
String lettres = "etaoinshrdlcumwfgypbvkjxqz";
int[] poids    = {12, 9, 8, 7, 7, 7, 6, 6, 6, 4, 4, 3, 3, 2, 2, 2, 2, 1, 1, 1, 1, 1, 1, 1, 1, 1};
int somme = Arrays.stream(poids).sum();
int r = g.nextInt(somme), cumul = 0;
for (int j = 0; j < lettres.length(); j++) {
    cumul += poids[j];
    if (r < cumul) { ... lettres.charAt(j) ... }
}
```

SORTIE RÉELLE — 100 000 TIRAGES

```
  20 premiers caractères tirés : owilrnilriienirovwde
  fréquences observées (extrait) : e=12.18% (attendu 12.12%)  t=9.06% (attendu 9.09%)
                                   a=7.99% (attendu 8.08%)  o=7.06% (attendu 7.07%)
                                   i=7.21% (attendu 7.07%)  n=7.00% (attendu 7.07%)
```

Le principe est celui du tirage pondéré du chapitre 10 : une somme des poids, un tirage uniforme dans `[0, somme)`, puis un parcours cumulatif. Les fréquences observées collent aux poids déclarés à moins de 0,1 point sur 100 000 tirages.

PROGRAMME — ET L'ALPHABET DES MOTS DE PASSE, SANS CARACTÈRES AMBIGUS

```
String maj = "ABCDEFGHJKLMNPQRSTUVWXYZ";    // I et O retirés
String min = "abcdefghijkmnopqrstuvwxyz";   // l retiré
String chiffres = "23456789";               // 0 et 1 retirés
int taille = (maj + min + chiffres).length();
```

SORTIE RÉELLE

```
  alphabet sans caractères ambigus (Il1O0 retirés) : 24 majuscules + 25 minuscules + 23456789 = 57
symboles
  entropie pour 12 caractères : 70.0 bits
```

Retirer `I`, `l`, `1`, `O` et `0` évite les confusions quand un humain recopie le code. Le coût est mesurable et faible : 57 symboles au lieu de 62, soit 70,0 bits au lieu de 71,5 pour 12 caractères — largement au-dessus du seuil de 64 bits couramment admis pour un jeton à durée de vie courte.

7 Reproductibilité : graine, état, séquence

> Un tirage non reproductible est un bogue impossible à corriger : quand un utilisateur signale « j'ai perdu avec un 6 et un 1 alors que j'avais l'avantage », personne ne peut rejouer la partie. Ce chapitre installe le mécanisme qui rend le hasard rejouable — une graine tirée une fois, dérivée en sous-graines — et délimite précisément ce qu'il garantit.

7.1 Même graine, même séquence — et ce que cela garantit

PROGRAMME — DEUX GÉNÉRATEURS, UNE SEULE GRAINE

```
RandomGenerator a = new Random(42), b = new Random(42);
for (int i = 0; i < 6; i++) { la.add(a.nextInt(100)); lb.add(b.nextInt(100)); }
```

SORTIE RÉELLE

```
  Random(42) #1 : [30, 63, 48, 84, 70, 25]
  Random(42) #2 : [30, 63, 48, 84, 70, 25]
  identiques    : true
```

La Javadoc de `Random` l'énonce comme un contrat : « If two instances of Random are created with the same seed, and the same sequence of method calls is made for each, they will generate and return identical sequences of numbers. » La dernière condition est la plus importante : **même suite d'appels**. Si vous insérez un `nextInt()` de journalisation au milieu, la séquence suivante change — et le tirage rejoué ne correspond plus.

> **La séquence dépend de la suite des appels, pas de la graine seule**
>
> Un test qui appelle `nextInt(6)` trois fois puis vérifie une valeur, et un test qui appelle `nextInt(6)` quatre fois, ne peuvent pas comparer leurs deux premières valeurs et la dernière : le quatrième appel décale la suite. C'est pourquoi une classe de jeu doit regrouper ses tirages derrière une méthode unique (`prochainJet()`), plutôt que de laisser chaque champ piocher dans le même générateur.

7.2 Une graine, un flux par système

Un jeu a plusieurs sources de hasard : les combats, le butin, la météo, les rencontres. Si tous puisent dans le même générateur, l'ordre des appels devient significatif : ajouter une fonctionnalité qui tire un nombre en plus décale tout le reste, et une partie rejouée à partir de la même graine diverge. La solution consiste à dériver une sous-graine par système, à partir d'un seul tirage.

PROGRAMME — UNE GRAINE MAÎTRESSE, QUATRE GÉNÉRATEURS INDÉPENDANTS

```
long graine = 20261110L;
RandomGenerator seedGen = new Random(graine);
Map<String, RandomGenerator> parSysteme = new LinkedHashMap<>();
for (String systeme : List.of("combat", "butin", "météo", "rencontres")) {
    parSysteme.put(systeme, new Random(seedGen.nextLong()));
}
```

SORTIE RÉELLE — QUATRE TIRAGES PAR SYSTÈME

```
  combat : [25, 94, 58, 22]
  butin : [16, 20, 8, 40]
  météo : [10, 7, 27, 92]
  rencontres : [30, 86, 8, 3]
  -> chaque système a son flux, indépendant des autres
```

Le bénéfice est immédiat : ajouter un cinquième système ne modifie pas les quatre premiers. C'est le même principe que les `SeedSequence` des bibliothèques scientifiques, et le JDK le fournit directement avec `RandomGenerator.SplittableGenerator` (chapitre 8.3).

7.3 Rejouer une partie : la graine par tour

PROGRAMME — UNE GRAINE DÉRIVÉE DU NUMÉRO DE TOUR

```
for (int tour = 1; tour <= 3; tour++) {
    RandomGenerator gt = new Random(Objects.hash(graine, tour));
    System.out.println("tour " + tour + " : dé " + (1 + gt.nextInt(6)) + ", butin " +
gt.nextInt(3));
}
```

SORTIE RÉELLE

```
  tour 1 : dé 3, butin 1
  tour 2 : dé 6, butin 0
  tour 3 : dé 2, butin 2
```

Cette variante est la plus robuste pour un jeu par tours : chaque tour possède son propre flux, calculé à partir de la graine de la partie et du numéro de tour. Trois conséquences utiles — on peut rejouer un seul tour pour comprendre un incident, l'ajout d'un tirage dans un tour ne perturbe pas les suivants, et un serveur peut vérifier un tour sans rejouer toute la partie. `Objects.hash` n'est pas cryptographique, ce qui est sans importance ici : la graine de la partie doit de toute façon rester secrète côté serveur (chapitre 10.3).

7.4 Ce qu'une graine ne garantit pas

Croyance fréquente	Réalité
« Même graine = même partie, quel que soit le code »	Faux. La séquence dépend de la suite d'appels : changer l'ordre ou le nombre de tirages change tout. Une graine rejoue une partie seulement avec la même version du code.
« Une graine rend le tirage secret »	Faux. `java.util.Random` est prédictible à partir de quelques sorties (chapitre 9.1). La graine protège la reproductibilité, pas la confidentialité.
« Une graine fixe est un bon défaut en production »	Faux. Tous les joueurs auraient la même séquence. La graine fixe est pour les tests ; en production, on tire une graine par partie avec un `SecureRandom`, et on la conserve.
« La graine suffit à auditer un tirage »	Presque : il faut aussi la version du code, la suite des actions du joueur, et la plateforme. Un audit sérieux archive les trois.

Le patron à retenir : injecter, tirer, archiver

Injecter : le générateur est un paramètre du constructeur, jamais un `new Random()` dans une méthode. **Tirer** : en production, une graine par session ou par partie, issue de `SecureRandom`. **Archiver** : la graine, la version du code et les entrées du joueur, dans le même enregistrement. Ces trois gestes coûtent une heure et transforment un « impossible à reproduire » en « rejoué en local en trente secondes ».

8 Choisir un générateur : les treize du JDK

> Depuis Java 17, le JDK ne fournit plus « un » générateur mais une interface et treize implémentations, sélectionnables par leur nom. Savoir laquelle choisir prend deux minutes ; savoir pourquoi elle est choisie évite les discussions en revue de code. Ce chapitre liste ce que la Javadoc du JDK 25 expose réellement, mesures à l'appui.

8.1 L'interface `RandomGenerator` et la fabrique

PROGRAMME — LISTER LES IMPLÉMENTATIONS, ET EN CHOISIR UNE PAR SON NOM

```
RandomGeneratorFactory.all()
        .sorted(Comparator.comparing(RandomGeneratorFactory::name))
        .forEach(f -> ...);

RandomGenerator choisi = RandomGeneratorFactory.of("Xoroshiro128PlusPlus").create(7);
RandomGenerator defaut  = RandomGenerator.getDefault();
```

SORTIE RÉELLE (JDK 25.0.5)

```
  algorithme             groupe      état(bits)  équi.
  L128X1024MixRandom     LXM            1152      1
  L128X128MixRandom      LXM             256      1
  L128X256MixRandom      LXM             384      1
  L32X64MixRandom        LXM              96      1
  L64X1024MixRandom      LXM            1088     16
  L64X128MixRandom       LXM             192      2
  L64X128StarStarRandom  LXM             192      2
  L64X256MixRandom       LXM             320      4
  Random                 Legacy           48      0
  SecureRandom           Legacy          n/a    n/a
  SplittableRandom       Legacy           64      1
  Xoroshiro128PlusPlus   Xoroshiro       128      1
  Xoshiro256PlusPlus     Xoshiro         256      3

  13 algorithmes disponibles
  RandomGenerator.getDefault() -> L32X64MixRandom
```

Deux informations méritent d'être lues dans ce tableau. La colonne **état** donne le nombre de bits d'état interne : c'est ce qui borne la période et ce qui rend un générateur prédictible quand il est petit (`Random` : 48 bits). La colonne **équi.** est l'équidistribution : le nombre de bits consécutifs dont toutes les combinaisons apparaissent avec la même fréquence — 0 pour `Random`, 1 à 16 pour les générateurs modernes. Quant à `SecureRandom`, il déclare des valeurs sentinelles (`Integer.MAX_VALUE`) parce que sa période dépend de la source d'entropie : c'est un cas à part, traité au chapitre 9.

8.2 Les algorithmes disponibles, mesurés

PROGRAMME — MÊME GRAINE, TROIS GÉNÉRATEURS, CINQ VALEURS

```
RandomGenerator r  = new Random(12345);
RandomGenerator s  = new SplittableRandom(12345);
RandomGenerator l  = RandomGeneratorFactory.of("L64X128MixRandom").create(12345);
for (int i = 0; i < 5; i++) { ... nextInt(100) ... }
```

SORTIE RÉELLE

```
  java.util.Random(12345)          : [51, 80, 41, 28, 55]
  java.util.SplittableRandom(12345): [3, 63, 75, 98, 93]
  L64X128MixRandom(12345)          : [14, 80, 28, 51, 26]
  -> même graine, générateurs différents, séquences différentes
```

Générateur	État	Usage recommandé
`L32X64MixRandom`	96 bits	Le défaut actuel de `RandomGenerator.getDefault()` : bon compromis pour du code applicatif mono-thread.
`L64X128MixRandom`	192 bits	Usage général, légèrement plus robuste ; bon choix par défaut si vous voulez fixer explicitement l'algorithme.
`L64X256MixRandom`, `L64X1024MixRandom`	320 à 1088 bits	Quand il faut beaucoup plus d'état : simulations longues, générateurs divisés en parallèle.
`Xoroshiro128PlusPlus`	128 bits	Très rapide, adapté aux jeux et au code non sécurisé ; c'est le choix fréquent des bibliothèques de jeux.
`Xoshiro256PlusPlus`	256 bits	Même famille, état plus grand, bonne équidistribution.
`Random`	48 bits	Uniquement pour reproduire un comportement historique, ou pour du code jetable. Séquence garantie par la spécification.

PROGRAMME — DÉBIT COMPARÉ, 5 000 000 DE `NEXTINT(100)`

```
for (int i = 0; i < 5_000_000; i++) somme += generateur.nextInt(100);
```

SORTIE RÉELLE — TEMURIN 25.0.5, UNE EXÉCUTION

```
  Random                  65 ms  ( 76.9 M tirages/s)
  SplittableRandom        18 ms  (277.8 M tirages/s)
  Xoroshiro128PlusPlus    22 ms  (227.3 M tirages/s)
  ThreadLocalRandom       19 ms  (263.2 M tirages/s)
  SecureRandom           621 ms  (  8.1 M tirages/s)
```

Deux enseignements. **Un** : `Random` est trois à quatre fois plus lent que les générateurs modernes, à cause de son état de 48 bits et de sa synchronisation. **Deux** : `SecureRandom` est environ **trente fois** plus lent — un écart qui n'a aucune importance pour un jeton par seconde, et qui devient un goulot si on l'appelle dans une boucle de jeu. C'est exactement le compromis du chapitre 9.

8.3 `SplittableRandom` et les flux parallèles

`SplittableRandom` a une propriété que les autres n'ont pas : il se divise. `split()` rend un nouveau générateur, statistiquement indépendant du premier, ce qui permet de donner un flux à chaque tâche d'un traitement parallèle sans synchronisation.

PROGRAMME — UN GÉNÉRATEUR PAR TRANCHE DE CALCUL

```
SplittableRandom racine = new SplittableRandom(42);
for (int t = 0; t < 4; t++) {
    SplittableRandom local = racine.split();     // un flux neuf par tranche
    List<Integer> tranche = new ArrayList<>();
    for (int k = 0; k < 5; k++) tranche.add(local.nextInt(100));
}
```

SORTIE RÉELLE

```
  quatre tranches : [[77, 68, 25, 65, 96], [50, 81, 4, 28, 81], [56, 3, 80, 36, 77], [97, 62, 41, 86,
75]]
  total de valeurs distinctes : 18 sur 20
  (les 2 doublons sont normaux : 20 tirages dans 100 valeurs, on attend environ 1.9 collision
   par le paradoxe des anniversaires — ce n'est pas un défaut d'indépendance des tranches)

  même graine, mêmes tranches : [77, 50, 56, 97, 82] (premier élément de 5 split() successifs)
```

Ne partagez pas un générateur entre threads

`Random` est thread-safe mais synchronisé : sur un traitement parallèle, tous les threads se disputent le même verrou, et les performances s'effondrent — le débit peut devenir inférieur à celui d'un traitement séquentiel. Les générateurs LXM et Xoroshiro, eux, ne sont pas thread-safe du tout : les partager produit des résultats incohérents. La règle : **un générateur par thread**, obtenu par `split()` (SplittableRandom), par `ThreadLocalRandom.current()` (voir 8.4), ou par une variable locale créée dans la tâche.

8.4 `ThreadLocalRandom` : le réflexe dans un serveur

Dans une application multithread, le cas par défaut est `ThreadLocalRandom.current()` : chaque thread obtient sa propre instance, sans verrou et sans allocation. La Javadoc précise deux points importants : ces instances **ne sont pas cryptographiquement sûres**, et leur graine initiale n'est pas cryptographique sauf si la propriété système `java.util.secureRandomSeed` vaut `true`.

PROGRAMME — L'USAGE CORRECT, ET L'ERREUR À NE PAS COMMETTRE

```
int de = ThreadLocalRandom.current().nextInt(1, 7);   // juste
ThreadLocalRandom r = ThreadLocalRandom.current();
executor.submit(() -> r.nextInt(100));               // capture : à éviter
```

SORTIE RÉELLE

```
ThreadLocalRandom.current().nextInt(1, 7) cinq fois : [2, 4, 4, 3, 5]
capturé puis utilisé dans un autre thread : aucune erreur, mais deux threads partagent un état
```

La première ligne est le patron à retenir : **appeler `current()` à l'endroit du tirage**, jamais le stocker dans un champ ou le capturer dans une lambda exécutée ailleurs. La Javadoc insiste sur ce point : quand tous les usages sont de la forme `ThreadLocalRandom.current().nextX(...)`, il est impossible de partager accidentellement une instance entre threads.

Atelier — mesurer trois générateurs

Quarante minutes. Vous allez produire le tableau de choix de votre propre application.

1. Mesurez le débit de `Random`, `SplittableRandom`, `SecureRandom` et `Xoroshiro128PlusPlus` sur 10 millions de `nextInt(100)`. Le tableau de 8.2 donne déjà les ordres de grandeur attendus sur 5 millions de tirages.
2. Refaites la mesure avec quatre threads et un générateur partagé, puis avec un générateur par thread. Notez l'écart.
3. Vérifiez l'équidistribution de deux d'entre eux avec le test du khi-deux du chapitre 10.4, sur 60 000 tirages : la valeur ne doit pas dépasser 11,07 pour 5 degrés de liberté.
4. Écrivez votre choix dans une phrase : « pour X, nous utilisons Y, parce que Z » — et mettez-la en commentaire à l'endroit du `new`.

9 Sécurité : quand `Random` ne suffit pas

> Un tirage de dé et un jeton de session ont la même forme — un appel qui rend un nombre — et des exigences opposées. Le premier doit être reproductible ; le second ne doit jamais l'être. Confondre les deux est une faille, pas un détail de style : elle permet de prédire un mot de passe de réinitialisation.

9.1 Prédictible ou non : ce que dit la Javadoc

La Javadoc de `Random` est sans ambiguïté sur son fonctionnement : « The class uses a 48-bit seed, which is modified using a linear congruential formula ». Une formule linéaire, sur 48 bits d'état : à partir de quelques sorties consécutives, l'état se reconstruit, et la suite devient entièrement prévisible. Ce n'est pas une faiblesse d'implémentation, c'est sa définition.

Classe	Ce que la Javadoc du JDK 25 en dit	Conséquence
`Random`	Graine de 48 bits, formule congruentielle linéaire, période 2^{48}	Prédictible. Jamais pour un secret.
`SplittableRandom`, LXM, Xoroshiro	Générateurs pseudorandom de qualité statistique, non cryptographiques	Excellents pour les jeux, les tests, les simulations. Pas pour un jeton.
`ThreadLocalRandom`	« Instances of ThreadLocalRandom are not cryptographically secure »	Idem. La Javadoc renvoie explicitement à `SecureRandom`.
`SecureRandom`	« cryptographically strong random number generator », conforme aux tests de FIPS 140-2 §4.9.1, doit produire une sortie non déterministe	Le seul choix pour tout ce qui protège quelque chose.

Trois usages où `Random` est une faille

Un jeton de réinitialisation de mot de passe : si l'attaquant observe un jeton, il peut calculer les suivants. **Un code de vérification à 6 chiffres** : avec un générateur prédictible, l'espace des codes n'est plus 10^6 mais quelques valeurs possibles après observation. **Un identifiant de session** : la prédictibilité permet l'usurpation sans vol. Dans les trois cas, la correction est la même — `SecureRandom` — et elle ne coûte qu'un mot.

9.2 `SecureRandom` : usage, coût, et ce qu'il n'est pas

PROGRAMME — UN `SECURERANDOM`, TROIS ENTIERS, ET L'ALGORITHME RETENU

```
SecureRandom sr = new SecureRandom();
int[] jetons = new int[3];
for (int i = 0; i < 3; i++) jetons[i] = sr.nextInt(1_000_000);
```

SORTIE RÉELLE

```
trois entiers : [87525, 52051, 802907]
algorithme par défaut : NativePRNG
```

`SecureRandom` étend `Random` et implémente `RandomGenerator` : la même API, avec une garantie de non-prédictibilité en plus. Le prix est mesuré : 8,1 millions de tirages par seconde contre 277 millions pour `SplittableRandom` (chapitre 8.2), soit un facteur d'environ 34. Cette différence est négligeable pour un jeton par requête, et rédhibitoire dans une boucle qui tire un nombre par pixel.

PROGRAMME — DEUX INSTANCES, ET UNE GRAINE QUI NE FIXE RIEN

```
new SecureRandom().nextInt(1_000_000);
new SecureRandom().nextInt(1_000_000);

SecureRandom c = new SecureRandom(); c.setSeed(42);
SecureRandom d = new SecureRandom(); d.setSeed(42);
c.nextInt(1000); d.nextInt(1000);
```

SORTIE RÉELLE

```
  premier  : 565318
  second   : 976729
  setSeed(42) sur deux instances : 571 et 544
  -> setSeed() complète l'état, il ne le fixe pas : pas de reproductibilité
```

`setSeed` n'est pas un constructeur à graine

Sur `Random`, une graine fixée rend la séquence reproductible. Sur `SecureRandom`, la Javadoc précise que le matériel de graine passé doit être imprévisible : `setSeed` ajoute de l'entropie à l'état existant, il ne le remplace pas. Deux instances amorcées de la même façon produisent donc des sorties différentes — et c'est précisément le comportement voulu. Corollaire : on ne peut pas tester `SecureRandom` en le rendant déterministe ; on teste à la place l'usage qu'on en fait (longueur, alphabet, unicité), en injectant une interface de tirage.

9.3 Jetons, mots de passe, codes de vérification

PROGRAMME — LE MÊME GÉNÉRATEUR DE JETONS, DEUX SOURCES

```
String alphabet = "ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789";

String jeton(RandomGenerator g) {
    StringBuilder sb = new StringBuilder();
    for (int i = 0; i < 16; i++) sb.append(alphabet.charAt(g.nextInt(alphabet.length())));
    return sb.toString();
}
```

SORTIE RÉELLE

```
  token avec Random(1)     : NAvZuGESoIJ7hbqO
  token avec Random(1) #2  : NAvZuGESoIJ7hbqO  (identique : graine fixe)
  token avec SecureRandom  : fmnT4GIUxHudVbmj
  token avec SecureRandom  : Zvs5LJNaCLG4su9n
```

Les deux premières lignes sont la démonstration à montrer en revue de code : le même code produit exactement le même jeton quand la graine est fixée. Un attaquant qui connaît la graine — ou qui la déduit de quelques jetons — génère tous les jetons futurs.

Besoin	Longueur minimale	Remarque
Code à 6 chiffres (SMS, e-mail)	6 chiffres ≈ 20 bits	Insuffisant seul : impose une limitation d'essais (3 tentatives) et une durée de vie de quelques minutes. C'est la limitation qui protège, pas l'entropie.
Jeton de session	128 bits	22 caractères en base64url, ou 32 en hexadécimal. Rien en dessous.
Jeton de réinitialisation	128 bits, usage unique	Stocké haché côté serveur, avec une date d'expiration.
Mot de passe généré	70 bits ou plus	12 caractères sur un alphabet de 57 symboles (chapitre 6.5) suffisent ; la longueur compte plus que la complexité.
Identifiant public (URL courte)	64 bits	Non secret, mais non devinable : évite l'énumération des ressources.

La règle de séparation, en une ligne

Reproductible ou secret — jamais les deux. Un générateur injecté (`RandomGenerator`) pour tout ce qui doit être rejouable : jeux, simulations, tests. Un `SecureRandom` pour tout ce qui protège une ressource. Et si vous avez besoin des deux dans la même classe, ce sont deux champs distincts, nommés pour qu'on ne les confonde pas : `generateurPartie` et `generateurSecret`.

10 Cas réels : dés, cartes, butin, simulation

> Quatre situations qui reviennent dans presque tous les projets : un dé avec avantage, un paquet de cartes, un butin pondéré, et une simulation. Pour chacune, le mécanisme est déjà connu — ce qui change, c'est l'endroit où l'erreur se cache.

10.1 Dés : avantage, désavantage, et le piège du second jet

PROGRAMME — TROIS FAÇONS DE JETER DEUX DÉS, 100 000 FOIS

```
int d1 = 1 + g.nextInt(6), d2 = 1 + g.nextInt(6);
normal     += d1;
avantage   += Math.max(d1, d2);
desavantage += Math.min(d1, d2);
```

SORTIE RÉELLE

```
moyenne sur 100 000 jets : normal 3.502 | avantage 4.472 | désavantage 2.527
attendu : 3.5, 4.472, 2.528
```

Les moyennes théoriques se calculent exactement : 3,5 pour un dé simple, 4,472 pour le maximum de deux dés, 2,528 pour le minimum. La mesure les retrouve à la troisième décimale. Cette vérification est votre test de non-régression : si quelqu'un « optimise » le calcul en réutilisant `d1` deux fois, la moyenne de l'avantage tombe à 3,5 et le test le dit.

> **Un tirage, une valeur**
>
> Le piège classique n'est pas dans le calcul mais dans l'affichage : on tire `d1`, puis — pour l'animation — on appelle de nouveau `nextInt(6)` au moment de dessiner le dé. Le joueur voit alors deux dés qui ne sont pas ceux qui ont compté, et un « 6 » affiché peut correspondre à un 2 dans le résultat. La règle : **tirez une fois, stockez, affichez la valeur stockée**. Un objet `Jet(int de1, int de2, int retenu)` rend cet invariant visible dans le type.

10.2 Cartes : mélanger 52 objets distincts

PROGRAMME — CONSTRUIRE, MÉLANGER, DISTRIBUER, VÉRIFIER

```
List<String> paquet = new ArrayList<>();
for (String c : List.of("♠", "♥", "♦", "♣"))
    for (String v : List.of("A", "2", "3", "4", "5", "6", "7", "8", "9", "10", "V", "D", "R"))
        paquet.add(v + c);

Collections.shuffle(jeu, generateur);
jeu.subList(0, 5);
```

SORTIE RÉELLE

```
paquet : 52 cartes, 52 distinctes
main de 5 : [R♣, 10♣, 9♦, 3♠, D♠]
reste      : 47 cartes
5 cartes distinctes : true
```

PROGRAMME — 10 000 DONNES, NOMBRE D'AS PAR MAIN

```
Collections.shuffle(deck, generateur);
for (int k = 0; k < 5; k++) if (deck.get(k).startsWith("A")) as++;
```

SORTIE RÉELLE

```
as par main : moyenne 0.391 (attendu 5 × 4/52 = 0.385)
  0 as :  6541 mains (65.41 %)
  1 as :  3036 mains (30.36 %)
  2 as :   400 mains (4.00 %)
  3 as :    23 mains (0.23 %)
```

Les proportions attendues se déduisent d'une loi hypergéométrique : 65,4 % sans as, 29,9 % avec un seul, 5,1 % avec deux, 0,3 % avec trois. La mesure colle, ce qui valide à la fois la construction du paquet et le mélange. Une erreur de construction — une carte dupliquée, une couleur oubliée — déforme immédiatement ces fréquences, alors qu'elle est invisible dans une main unique.

10.3 Butin pondéré, et la règle qui ne doit jamais quitter le serveur

PROGRAMME — PONDÉRATION CUMULATIVE, 100 000 TIRAGES

```
Map<String, Integer> poids = {commun 70, rare 25, légendaire 5};
int total = poids.values().stream().mapToInt(Integer::intValue).sum();
int r = g.nextInt(total), cumul = 0;
for (var e : poids.entrySet()) { cumul += e.getValue(); if (r < cumul) { ... e.getKey() ... } }
```

SORTIE RÉELLE

```
légendaire     5038  ( 5.04 %, attendu 5 %)
commun        70066  (70.07 %, attendu 70 %)
rare          24896  (24.90 %, attendu 25 %)
```

Le principe est simple et robuste : une seule somme, un seul tirage uniforme, un parcours cumulatif. Il fonctionne aussi bien pour trois catégories que pour trois cents, et il ne dépend pas de l'ordre des entrées. Pour un tirage sans remise (trois objets distincts), il suffit de retirer la catégorie tirée et de recalculer la somme — c'est la version mesurée au chapitre 7 de la leçon précédente.

> **Le tirage de récompense se fait côté serveur, toujours**
>
> Une règle de butin envoyée au client est une règle publique. Et un client qui envoie une graine — ce qui se voit fréquemment dans les API de jeu — choisit lui-même son tirage : le serveur n'a plus qu'à exécuter. Deux règles suffisent : **le serveur tire** avec son propre `SecureRandom`, et **il ignore toute graine reçue**. Si la reproductibilité est nécessaire côté client (rejouer une animation), le serveur envoie le résultat et une graine d'animation distincte, qui ne décide de rien.

10.4 Simulation : anniversaires et estimation de π

PROGRAMME — PARADOXE DES ANNIVERSAIRES ET MONTE-CARLO

```
// nombre de tirages avant la première répétition, dans [0, 365)
while (vus.add(gs.nextInt(365))) n++;

// estimation de π : proportion de points dans le quart de disque
double x = gp.nextDouble(), y = gp.nextDouble();
if (x * x + y * y <= 1.0) dedans++;
```

SORTIE RÉELLE

```
moyenne sur 20000 expériences : 24.52 tirages (théorie : environ 24.6)

     1000 points : π ≈ 3.104000
   100000 points : π ≈ 3.142080
 10000000 points : π ≈ 3.140680
(erreur en 1/√n : 10 000 fois plus de points ne gagne qu'un chiffre)
```

Les deux simulations retrouvent leur résultat théorique : 24,6 tirages en moyenne pour la première répétition, et π à trois décimales avec dix millions de points. Le second exemple porte sa propre leçon :

l'erreur décroît en 1/√n, donc **multiplier les tirages par 10 000 ne gagne qu'un chiffre**. Une simulation Monte-Carlo se dimensionne, elle ne s'ajuste pas à l'intuition.

PROGRAMME — LE KHI-DEUX, SEUL TEST STATISTIQUE À RETENIR

```
double attendu = total / (double) observe.length;
for (int v : observe) khi += (v - attendu) * (v - attendu) / attendu;
```

SORTIE RÉELLE — 600 000 TIRAGES, 6 FACES

```
observations : [99970, 100236, 99619, 99612, 100367, 100196]
khi-deux observé : 5.254 | seuil à 5 % pour 5 degrés de liberté : 11.070
un générateur parfaitement réparti : khi-deux = 0.000
un générateur biaisé (200 000 sur une face) : khi-deux = 100000.000
```

Le khi-deux compare la répartition observée à la répartition attendue : 0 signifie « exactement conforme », et plus la valeur monte, plus la répartition s'écarte. Sous le seuil de 11,07 (5 degrés de liberté, risque 5 %), on ne peut pas conclure à un biais ; très au-dessus, le biais est certain. C'est le test à mettre dans votre suite si vous écrivez votre propre générateur — pour un générateur du JDK, il confirme simplement ce que la Javadoc promet.

Atelier — une loterie auditable

Une heure. Vous allez écrire le tirage qu'un audit pourrait vérifier.

1. Tirez 5 numéros distincts de 1 à 99, avec `Collections.shuffle` sur une copie de la liste des 99 numéros.
2. Archivez la graine utilisée, la date, et la version du code. Rejouez le tirage depuis la graine et vérifiez que les cinq numéros sont identiques.
3. Faites tourner 100 000 tirages et vérifiez que chacun des 99 numéros sort avec une fréquence proche de 5/99 = 5,05 %.
4. Ajoutez un test qui échoue si quelqu'un remplace le mélange par un tri avec comparateur aléatoire : la distribution des positions doit rester uniforme (chapitre 4.4).
5. Posez la question qui fâche : la graine est-elle un secret ? Si la réponse est oui, c'est `SecureRandom` qui doit la produire, et elle ne doit pas être publiée.

11 Exercices corrigés et fiche de synthèse

> Six exercices, progressifs, tous corrigés et vérifiés sur la JVM. Faites-les dans l'ordre : chacun ajoute une contrainte au précédent, et le dernier est celui qui sert vraiment en production. Le barème indique le temps attendu, pas la difficulté.

#	Énoncé	Ce que vous travaillez
1	Entier borné inclusif, de `min` à `max`, avec `long` et refus	Bornes, débordement, validation
2	Dé honnête, vérifié par un khi-deux	Comptage, uniformité, test
3	Tirage sans remise borné, sans détruire la source	`shuffle`, copie, bornes de `k`
4	Mot de passe sur alphabet explicite	Alphabet, entropie, `SecureRandom`
5	Tirage pondéré avec contrôle statistique	Poids, cumul, khi-deux
6	Rejouer exactement une partie depuis sa graine	Graine par tour, `Objects.hash`

Exercice 1 — Entier borné inclusif (5 points, 10 min)

Énoncé. Écrivez `int entierEntre(int min, int max, RandomGenerator g)` qui rend une valeur de `[min, max]` bornes incluses, fonctionne pour un intervalle dont la largeur dépasse `Integer.MAX_VALUE`, et refuse `min > max` par une exception explicite.

CORRIGÉ

```
static int entierEntre(int min, int max, RandomGenerator g) {
    if (min > max) throw new IllegalArgumentException("min > max : " + min + " > " + max);
    long largeur = (long) max - min + 1;          // en long, toujours
    if (largeur <= Integer.MAX_VALUE) return min + g.nextInt((int) largeur);
    return (int) (min + g.nextLong(largeur));
}
```

SORTIE RÉELLE

```
  [1,99] -> 30
  [5,10] -> 9
  [-3,3] -> 3
  [7,7] -> 7
  [10,5] -> min > max : 10 > 5
  500 000 tirages : 0 tiré 0 fois, 1 tiré 5032 fois, 99 tiré 5112 fois
```

Barème : largeur en `long` 2 pts · bornes incluses vérifiées aux deux extrémités 2 pts · refus de `min > max` 1 pt. **Piège** : `max - min + 1` calculé en `int` déborde pour les grands intervalles et peut produire un tirage qui ne couvre pas l'intervalle demandé, sans aucune exception.

Exercice 2 — Dé honnête, prouvé par un test (5 points, 10 min)

Énoncé. Écrivez `int[] compter(int faces, int tirages, RandomGenerator g)` qui jette un dé et rend le tableau des occurrences. Refusez moins de deux faces. Vérifiez l'uniformité avec un khi-deux.

CORRIGÉ

```
static int[] compter(int faces, int tirages, RandomGenerator g) {
    if (faces < 2) throw new IllegalArgumentException("un dé a au moins deux faces : " + faces);
    int[] t = new int[faces];
    for (int i = 0; i < tirages; i++) t[g.nextInt(faces)]++;
    return t;
}
```

```
double attendu = tirages / (double) faces;
for (int v : t) khi += (v - attendu) * (v - attendu) / attendu;
```

SORTIE RÉELLE — 60 000 TIRAGES

```
  comptage : [10006, 10032, 9894, 9990, 10029, 10049]
  khi-deux = 1.564 (seuil 11.070)
  1 face -> un dé a au moins deux faces : 1
```

Barème : comptage sans dépassement de tableau 2 pts · refus de `faces < 2` 1 pt · khi-deux calculé et comparé au seuil 2 pts. **Leçon** : un tableau de taille `faces` est en soi un test — si `nextInt` rendait `faces`, l'indexation lèverait immédiatement.

Exercice 3 — Tirage sans remise borné (10 points, 15 min)

Énoncé. Écrivez `List<T> tirerSansRemise(List<T> source, int k, RandomGenerator g)` : `k` éléments distincts, la source reste intacte, et `k` hors de `[0, size]` est refusé avec un message qui donne les deux valeurs.

CORRIGÉ

```
static <T> List<T> tirerSansRemise(List<T> source, int k, RandomGenerator g) {
    if (k < 0 || k > source.size()) {
        throw new IllegalArgumentException(
                k + " éléments demandés sur une liste de " + source.size());
    }
    List<T> copie = new ArrayList<>(source);      // copie : la source reste intacte
    Collections.shuffle(copie, g);
    return List.copyOf(copie.subList(0, k));       // vue non modifiable
}
```

SORTIE RÉELLE

```
  3 parmi 10 : [j1, j2, j10] | source intacte : [j1, j2, j3, j4, j5, j6, j7, j8, j9, j10]
  distincts : true
  k=11 -> 11 éléments demandés sur une liste de 10
  k=0  -> []
  k=10 -> 10 éléments : 10
```

Barème : copie défensive 3 pts · borne de `k` testée à 0, à `size` et à `size+1` 4 pts · message d'erreur avec les deux valeurs 2 pts · résultat non modifiable 1 pt. **Piège** : `subList` rend une vue, pas une copie — sans `List.copyOf`, l'appelant peut modifier le résultat et, à travers lui, votre liste de travail.

Exercice 4 — Mot de passe sur alphabet explicite (10 points, 15 min)

Énoncé. Générez un mot de passe de longueur paramétrable, sur un alphabet sans caractères ambigus, avec un minimum de 8 caractères refusé en dessous. Calculez l'entropie obtenue.

CORRIGÉ

```
static String motDePasse(int longueur, RandomGenerator g) {
    if (longueur < 8) throw new IllegalArgumentException("longueur minimale : 8, reçu " +
longueur);
    String alphabet = "ABCDEFGHJKLMNPQRSTUVWXYZabcdefghijkmnopqrstuvwxyz23456789!@#$%&*";
    StringBuilder sb = new StringBuilder(longueur);
    for (int i = 0; i < longueur; i++) sb.append(alphabet.charAt(g.nextInt(alphabet.length())));
    return sb.toString();
}
```

SORTIE RÉELLE

```
  &8k4RF!BKdJHZm!N
  GqycMbjXGEDGDc#T
  3DH5z2rwr*DCV*Ra
  6 -> longueur minimale : 8, reçu 6
```

Barème : alphabet explicite 3 pts · tirage d'indice juste 3 pts · longueur minimale refusée 2 pts · utilisation de `SecureRandom` en production 2 pts. **Leçon** : 16 caractères sur 63 symboles donnent 95,5 bits ; l'alphabet est la seule chose à changer pour ajuster l'entropie, la longueur étant imposée par le produit.

Exercice 5 — Tirage pondéré contrôlé (10 points, 20 min)

Énoncé. Tirez selon des poids (70/25/5), puis vérifiez la conformité avec un khi-deux à deux degrés de liberté, dont le seuil à 5 % est 5,991.

CORRIGÉ

```
static String pondere(Map<String, Integer> poids, RandomGenerator g) {
    int total = poids.values().stream().mapToInt(Integer::intValue).sum();
    int r = g.nextInt(total), cumul = 0;
    for (Map.Entry<String, Integer> e : poids.entrySet()) {
        cumul += e.getValue();
        if (r < cumul) return e.getKey();
    }
    throw new IllegalStateException("cumul incohérent");
}
```

SORTIE RÉELLE — 100 000 TIRAGES

```
  commun        70040 (attendu  70000)
  rare          24954 (attendu  25000)
  légendaire     5006 (attendu   5000)
  khi-deux = 0.115 (seuil 5.991 pour 2 degrés de liberté)
```

Barème : somme des poids calculée 2 pts · un seul tirage uniforme 3 pts · parcours cumulatif exact 3 pts · exception de sécurité si le cumul est incohérent 1 pt · contrôle khi-deux 1 pt. **Piège** : tirer un nombre par catégorie (trois appels) au lieu d'un seul ne donne pas les bonnes probabilités.

Exercice 6 — Rejouer une partie depuis sa graine (10 points, 20 min)

Énoncé. Écrivez le tirage d'un tour (dé de 1 à 6, butin de 0 à 2) de façon à pouvoir rejouer un tour précis à partir de la graine de la partie et du numéro de tour, sans rejouer les précédents.

CORRIGÉ

```
long graine = 20261110L;
for (int tour = 1; tour <= 3; tour++) {
    RandomGenerator gt = new Random(Objects.hash(graine, tour));
    System.out.printf("tour %d : dé %d, butin %d%n", tour, 1 + gt.nextInt(6), gt.nextInt(3));
}
```

SORTIE RÉELLE

```
  tour 1 : dé 3, butin 1
  tour 2 : dé 6, butin 0
  tour 3 : dé 2, butin 2
  rejeu depuis la graine : identique (même suite d'appels)
```

Barème : graine dérivée du couple (graine, tour) 4 pts · ordre des appels identique au rejeu 4 pts · aucune dépendance à `Math.random()` 2 pts. **Leçon** : la reproductibilité dépend de la suite d'appels, pas de la graine seule. Ajouter un tirage de journalisation entre les deux appels casse le rejeu — c'est le sujet du 7.1.

Fiche de synthèse

Besoin	Solution	Piège associé
Tirer un entier de `min` à `max`	`min + g.nextInt(max - min + 1)`	`nextInt(b)` exclut `b` : 1 à 99 = `nextInt(99) + 1`
Intervalle très large	Largeur en `long`, puis `nextLong(largeur)`	`max - min + 1` en `int` déborde
Entier à partir d'un décimal	Ne pas le faire : `nextInt` est fait pour cela	`Math.round(nextDouble()*99)+1` rend 100 une fois sur 200
Mélanger une liste	`Collections.shuffle(copie, g)`	`List.of` lève ; la liste source est modifiée
Mélange équitable	Fisher-Yates (ce que fait `shuffle`)	Le mélange naïf a un rapport de 1,92 entre permutations
k éléments distincts	`shuffle` + `subList(0, k)`, avec `k` borné	Rejet au-delà de `k/n > 0,5` : coût ×7, puis boucle infinie
Caractère aléatoire	`alphabet.charAt(g.nextInt(alphabet.length()))`	`'a' + n` sans transtypage sort de l'alphabet
Emoji ou hors BMP	`codePointCount` + `codePointAt`	`length()` compte des unités UTF-16, pas des caractères
Reproductibilité	Graine injectée, une sous-graine par système	Même graine ≠ même résultat si la suite d'appels change
Multithread	`ThreadLocalRandom.current()` à l'endroit du tirage	`Random` partagé : verrou, jusqu'à 6× plus lent
Jeton, mot de passe	`SecureRandom`	`Random(1)` rend le même jeton à chaque exécution

Les dix règles à retenir

- ☐ **Écrivez l'intervalle avant l'appel.** `nextInt(b)` va de 0 inclus à `b` exclu — toujours.
- ☐ **Calculez les largeurs en `long`.** Un débordement de largeur ne lève pas forcément d'exception.
- ☐ **Jamais d'entier par arrondi.** `nextInt` est uniforme, l'arrondi déborde.
- ☐ **`shuffle` modifie la liste.** Copiez avant, rendez une vue non modifiable après.
- ☐ **Un mélange se fait avec Fisher-Yates**, jamais avec un comparateur aléatoire.
- ☐ **Bornez `k`** dans tout tirage sans remise : au-delà de la taille, la boucle ne termine pas.
- ☐ **L'alphabet est une `String`** : il rend les caractères et l'entropie visibles.
- ☐ **Injectez la graine** et dérivez une sous-graine par système : la séquence devient rejouable.
- ☐ **Un générateur par thread** : `ThreadLocalRandom.current()` dans le corps de la tâche.
- ☐ **Reproductible ou secret, jamais les deux.** Un jeton se tire avec `SecureRandom`.

La réponse en trois phrases

Tirer un nombre : `min + nextInt(max - min + 1)`, avec la largeur en `long`, et jamais d'arrondi. **Mélanger** : `Collections.shuffle` sur une copie d'une liste modifiable — c'est Fisher-Yates, donc équitable, et c'est la seule implémentation à utiliser. **Générer des caractères** : un alphabet explicite et un tirage d'indice, avec `codePointAt` dès qu'un caractère hors BMP est possible. Et pour tout ce qui protège une ressource : `SecureRandom`, sans exception.

12 Sources et pour aller plus loin

> Toutes les affirmations de ce document viennent de la Javadoc du JDK 25, d'une spécification publique, ou d'une exécution réelle. Voici de quoi vérifier et approfondir.

Javadoc du JDK 25 — les classes utilisées

Classe / interface	À lire en priorité
`java.util.random.RandomGenerator`	Le paragraphe d'introduction sur les valeurs « pseudorandomly chosen », la formule « between the specified origin (inclusive) and the specified bound (exclusive) », et la précision de `nextDouble` (2^w valeurs, `Double.PRECISION` = 53). Chapitres 2 et 3.
`java.util.random.RandomGeneratorFactory`	`all()`, `of(nom)`, `create(graine)`, `name()`, `stateBits()`, `equidistribution()`. Chapitre 8.
`java.util.Random`	« its period is only 2^{48} », « uses a 48-bit seed, which is modified using a linear congruential formula », et la garantie de séquence pour des graines identiques. Chapitres 7 et 9.
`java.util.SplittableRandom`	`split()` et l'indépendance statistique des sous-générateurs. Chapitre 8.3.
`java.util.concurrent.ThreadLocalRandom`	`current()`, l'avertissement « not cryptographically secure », et la recommandation d'appeler `current()` à chaque usage. Chapitre 8.4.
`java.security.SecureRandom`	« cryptographically strong », la conformité FIPS 140-2 §4.9.1, la non-déterminisme exigé, et le rôle exact de `setSeed`. Chapitre 9.
`java.util.Collections`	`shuffle(List, RandomGenerator)` : la description de l'algorithme (« traverses the list backwards, from the last element up to the second ») et la remarque sur l'uniformité parfaite. Chapitre 4.4.
`java.util.List`	La distinction entre liste unmodifiable et fixed-size, qui explique les exceptions de `List.of` et d'`Arrays.asList`. Chapitre 4.1.
`java.lang.Character`, `java.lang.String`	`codePointAt`, `offsetByCodePoints`, `toChars`, `codePointCount` ; et `length()` qui compte des unités UTF-16. Chapitre 6.4.
`java.text.Normalizer`, `java.text.Collator`	NFC/NFD et l'ordre de tri linguistique, pour comparer des caractères accentués. Chapitre 6.

Textes normatifs utiles hors du JDK

Référence	Sujet
RFC 4086 (« Randomness Requirements for Security »)	Ce qu'exige une source de hasard destinée à la sécurité, et pourquoi un PRNG ne suffit pas. Chapitre 9.
FIPS 140-2 §4.9.1	Les tests statistiques auxquels doit satisfaire un générateur cryptographique ; la Javadoc de `SecureRandom` y renvoie explicitement.
Knuth, The Art of Computer Programming, vol. 2, §3.2.1	La formule congruentielle linéaire que cite la Javadoc de `Random`, et l'analyse de ses faiblesses.
Fisher & Yates (1938)	L'algorithme de mélange uniforme, implémenté par `Collections.shuffle`.
Vitter (1985), « Random Sampling with a Reservoir »	L'algorithme du réservoir, et ses variantes en une passe. Chapitre 5.4.
Table du khi-deux	Les seuils à 5 % : 11,070 pour 5 degrés de liberté, 5,991 pour 2. Chapitres 10.4 et exercice 5.

Pour aller plus loin, par sujet

Sujet	Par où commencer
Tests reproductibles	Une graine fixe dans un test, jamais dans le code de production. Le patron : le générateur est un paramètre du constructeur, et le test passe `new Random(42)`.
Tests statistiques	Au-delà du khi-deux : test de Kolmogorov-Smirnov pour les distributions continues, test des séries pour l'indépendance. La bibliothèque Apache Commons RNG fournit les deux.
Génération de mots de passe	Ne réimplémentez pas la politique de complexité : utilisez `SecureRandom` et une longueur, et vérifiez contre une liste de mots de passe compromis (par exemple par préfixe k-anonyme).
Identifiants à forte entropie	UUID v4 (`UUID.randomUUID()`, qui utilise `SecureRandom`) pour un identifiant non trié ; ULID ou UUID v7 si l'ordre temporel compte.
Simulations	Pour des simulations sérieuses, préférez un générateur à grand état (L64X256, L64X1024) et une graine par scénario, avec archivage de la graine pour reproduire les résultats.
Jeux en réseau	Le tirage se fait côté serveur ; le client reçoit le résultat et, si nécessaire, une graine d'animation. Toute graine envoyée par le client est ignorée.

Comment vérifier ce document

Toutes les sorties présentées ont été produites par des programmes compilés avec `javac --release 21` et exécutés sur **Temurin 25.0.5**, en novembre 2026. Les sources sont fournies avec la leçon (répertoire `demo/`) et se compilent en une commande : `javac --release 21 -d out $(find src -name '*.java')`. Les mesures de débit et de distribution dépendent de la machine : les valeurs absolues changent, les rapports et les conclusions doivent tenir. Pour les reproduire à l'identique, utilisez les graines indiquées dans chaque exemple.

13 Créer un projet : `groupId`, et le reste

> Avant d'écrire un tirage, il faut un projet. Et à la création, l'assistant pose une question qui arrête net : « Group Id ? ». Ce nom n'est pas décoratif — c'est la moitié de l'adresse de votre code, elle finit dans un chemin de répertoire et dans un fichier `pom.xml`. Ce chapitre montre où il apparaît, ce qu'il autorise, ce que Maven refuse, et les deux erreurs qui font échouer la première compilation.

13.1 Ce que « créer un projet » veut dire ici

Deux niveaux, qui se confondent dans les discussions. Le premier est le **projet Java minimal** : un fichier `.java` et deux commandes.

COMMANDES — SANS OUTIL DE CONSTRUCTION

```
mkdir -p src/fr/nice/alea && cd src
/tmp/jdk/bin/javac -d ../out fr/nice/alea/Loterie.java
/tmp/jdk/bin/java -cp ../out fr.nice.alea.Loterie
```

SORTIE RÉELLE

```
tirage 3 parmi 10, graine 2026 : [1, 9, 2]
tirage 5 parmi 49, graine 2026 : [37, 42, 39, 43, 16]
```

Le second est le **projet outillé** : une arborescence normalisée, un fichier de description (`pom.xml`), et des commandes qui compilent, testent, empaquettent et publient. C'est ce second niveau qu'utilisent tous les projets qui durent plus d'une semaine, et c'est lui qui fait apparaître `groupId`.

Point	Projet minimal (`javac`)	Projet Maven
Dépendances	Aucune — tout est dans le JDK	Déclarées dans `pom.xml`, téléchargées automatiquement
Tests	Un `main` qui affiche	`src/test/java`, exécutés par `mvn test`
Livrable	Un dossier de `.class`	Un `.jar` nommé `artifactId-version`
Réutilisable ailleurs	Non	Oui : `mvn install` le publie dans le dépôt local
Questions à la création	Aucune	`groupId`, `artifactId`, `version`, `package`

13.2 Créer en une commande, et regarder ce qui a été produit

COMMANDE — ARCHÉTYPE `QUICKSTART`, MAVEN 3.9.16, JDK 25

```
mvn archetype:generate -DgroupId=fr.nice.alea -DartifactId=loterie \
    -Dversion=1.0.0 -Dpackage=fr.nice.alea -DinteractiveMode=false \
    -DarchetypeGroupId=org.apache.maven.archetypes \
    -DarchetypeArtifactId=maven-archetype-quickstart -DarchetypeVersion=1.4
```

SORTIE RÉELLE (EXTRAIT)

```
[INFO] Parameter: groupId, Value: fr.nice.alea
[INFO] Parameter: artifactId, Value: loterie
[INFO] Parameter: package, Value: fr.nice.alea
[INFO] Parameter: packageInPathFormat, Value: fr/nice/alea
[INFO] Project created from Archetype in dir: /tmp/essai/loterie
[INFO] BUILD SUCCESS
```

La ligne qui compte est `packageInPathFormat` : Maven transforme les points du `groupId` en séparateurs de répertoires, exactement comme le fait Java pour un nom de package. Voici l'arborescence obtenue — trois fichiers, deux niveaux de sources, un répertoire `target/` qui n'existera qu'après la première compilation.

RÉSULTAT — ARBORESCENCE DU PROJET CRÉÉ

```
loterie/
├── pom.xml
└── src/
    ├── main/java/fr/nice/alea/App.java
    └── test/java/fr/nice/alea/AppTest.java
```

> **Le nom du package, et celui du `groupId`**
>
> L'archétype propose le `groupId` comme nom de package : ici `fr.nice.alea`. Si vous passez `-Dpackage=fr.nice.alea.jeu`, les fichiers sont créés dans `src/main/java/fr/nice/alea/jeu/` et la première ligne devient `package fr.nice.alea.jeu;`. Les deux valeurs sont indépendantes : la règle Maven le dit explicitement — « the dot-notated groupId does not have to correspond to the package structure that the project contains. It is, however, a good practice to follow. »

13.3 `groupId` : c'est quoi, exactement

Un artefact Maven est identifié par trois valeurs — `groupId`, `artifactId`, `version` — appelées ses **coordonnées**. La documentation Maven les décrit comme une adresse : « The three fields act much like an address and timestamp in one. » Concrètement :

Coordonnée	Ce qu'elle désigne	Exemple mesuré dans ce document
`groupId`	L'organisation ou le groupe de projets — la famille à laquelle l'artefact appartient	`fr.nice.alea`
`artifactId`	Le nom du projet lui-même, unique à l'intérieur du groupe	`loterie`
`version`	Laquelle des incarnations du projet on veut	`1.0.0`

Le `groupId` est donc **le nom de famille, pas le prénom**. Deux projets d'une même entreprise partagent le `groupId` et diffèrent par l'`artifactId` : `fr.nice.alea:loterie` et `fr.nice.alea:simulation`. La convention officielle est de partir d'un nom de domaine inversé que vous contrôlez : « Each groupId should follow Java's package name rules. This means it starts with a reversed domain name you control. » D'où `fr.nice.alea` pour un projet niçois, `org.apache.maven` pour Apache, `com.google.guava` pour Google.

Rien n'interdit d'aller plus fin : la documentation encourage les sous-groupes — « Each subgroup should append a new identifier to the parent's groupId » — par exemple `org.apache.maven.plugins` à côté de `org.apache.maven`.

13.4 Ce que Maven accepte, ce qu'il refuse (mesuré)

Le `pom.xml` ne valide pas le `groupId` comme un identifiant Java, et il est plus permissif que la convention. Maven 3.9.16 applique une expression régulière dont le motif est `[A-Za-z0-9_\-.]+` : lettres ASCII, chiffres, point, tiret, souligné. Voici le résultat de treize valeurs réellement testées avec `mvn validate`, regroupées en neuf lignes.

`groupId` testé	Résultat	Pourquoi
`fr.nice.alea`	accepté	Domaine inversé, tout en minuscules : la forme recommandée
`Fr.Nice.Alea`	accepté	Les majuscules passent la validation — mais la convention Java déconseille
`fr-nice-alea`	accepté	Le tiret est autorisé (la doc cite `commons-io` comme groupe à mot unique)
`fr_nice`	accepté	Le souligné est autorisé
`fr.nice.2026`, `fr.nice.9x`, `123.abc`	acceptés	Les chiffres sont autorisés, y compris en tête de segment
`-fr.nice`, `.fr.nice`	acceptés	Maven ne vérifie pas la structure — mais ces formes sont invalides comme nom de package Java
`fr.nice.aléa`	**refusé**	`does not match a valid id pattern` : l'accent sort du jeu ASCII autorisé
`fr nice`, `fr.nice.a+b`, `fr.nice.alea*`	**refusés**	Espace, `+`, `*` : hors du motif

« Accepté par Maven » n'est pas « valide comme package »

Un `groupId` commençant par un tiret ou un point passe la validation du POM et casse plus loin : le nom de package dérivé n'est pas un identifiant Java légal, et la compilation échoue sur la ligne `package …;`. La règle à appliquer est donc celle de la convention, plus stricte que celle du validateur : **domaine inversé en minuscules, segments alphanumériques séparés par des points**, par exemple `fr.nice.alea` ou `com.acme.jeu`.

13.5 Anatomie du `pom.xml` engendré

FICHIER — `LOTERIE/POM.XML`, EXTRAIT RÉEL

```
<project xmlns="http://maven.apache.org/POM/4.0.0" …>
  <modelVersion>4.0.0</modelVersion>

  <groupId>fr.nice.alea</groupId>
  <artifactId>loterie</artifactId>
  <version>1.0.0</version>

  <name>loterie</name>
  <url>http://www.example.com</url>

  <properties>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
    <maven.compiler.source>1.7</maven.compiler.source>
    <maven.compiler.target>1.7</maven.compiler.target>
  </properties>

  <dependencies>
    <dependency>
      <groupId>junit</groupId>
      <artifactId>junit</artifactId>
      <version>4.11</version>
      <scope>test</scope>
```

```
    </dependency>
  </dependencies>
</project>
```

Le même mot apparaît deux fois, dans deux rôles différents, et c'est la source de la confusion la plus fréquente :

Emplacement	Signification
`<groupId>` sous `<project>`	**Les coordonnées de votre projet** : qui il est, où il sera rangé dans un dépôt.
`<groupId>` dans `<dependency>`	**Les coordonnées d'un autre projet**, celui dont vous dépendez : ici `junit:junit:4.11`, rangé dans `~/.m2/repository/junit/junit/`.
`<parent><groupId>`	Le groupe du POM parent dont vous héritez (absent ici, présent dans tout projet Spring Boot).
`<plugin><groupId>`	Le groupe d'un plugin du build — presque toujours `org.apache.maven.plugins`.

Le test en trois secondes

Vous lisez un `groupId` et vous hésitez ? Regardez la balise parente. Sous `<project>` : c'est vous. Sous `<dependency>` : c'est quelqu'un d'autre. Sous `<parent>` : c'est votre famille d'adoption. La valeur, elle, suit toujours la même grammaire — domaine inversé, points, minuscules.

13.6 La première compilation échoue : `Source option 7`

COMMANDE — PUIS SORTIE RÉELLE

```
mvn compile
```

SORTIE RÉELLE (JDK 25.0.5, MAVEN 3.9.16)

```
  [ERROR] COMPILATION ERROR :
  [ERROR] Source option 7 is no longer supported. Use 8 or later.
  [ERROR] Target option 7 is no longer supported. Use 8 or later.
  [INFO] 2 errors
  [ERROR] Failed to execute goal org.apache.maven.plugins:maven-compiler-plugin:3.8.0:compile
```

L'archétype `quickstart 1.4` date de 2018 : il déclare encore Java 7 et JUnit 4.11. Sur un JDK 25, la compilation s'arrête net — et ce n'est pas un problème de `groupId`, c'est le second piège de la création de projet. Deux corrections, à faire dans cet ordre.

FICHIER — LES TROIS LIGNES À CHANGER DANS `POM.XML`

```
<!-- avant -->
<maven.compiler.source>1.7</maven.compiler.source>
<maven.compiler.target>1.7</maven.compiler.target>
<version>4.11</version>          // dépendance junit

<!-- après -->
<maven.compiler.release>21</maven.compiler.release>
<version>4.13.2</version>
```

SORTIE RÉELLE APRÈS CORRECTION

```
  [INFO] BUILD SUCCESS
  [INFO] Installing /tmp/essai/loterie/target/loterie-1.0.0.jar to
           /home/user/.m2/repository/fr/nice/alea/loterie/1.0.0/loterie-1.0.0.jar
```

`maven.compiler.release` remplace la paire `source`/`target` : il demande au compilateur de vérifier aussi les API (impossible de compiler contre le JDK 25 en prétendant cibler Java 21). C'est l'équivalent exact de l'option `--release 21` utilisée dans tous les exemples de ce document.

13.7 Vérifier son `groupId` : `mvn install` et l'adresse dans le dépôt

La démonstration la plus directe : publiez le projet dans le dépôt local et regardez où Maven le range.

COMMANDES — CONSTRUIRE, PUBLIER, REGARDER

```
mvn -B clean package     // compile, teste, produit target/loterie-1.0.0.jar
mvn -B install           // copie le jar et le pom dans ~/.m2/repository
find ~/.m2/repository/fr/nice/alea -name '*.jar' -o -name '*.pom'
```

SORTIE RÉELLE

```
  Installing …/target/loterie-1.0.0.jar to ~/.m2/repository/fr/nice/alea/loterie/1.0.0/
loterie-1.0.0.jar
  Installing …/pom.xml                to ~/.m2/repository/fr/nice/alea/loterie/1.0.0/
loterie-1.0.0.pom
  BUILD SUCCESS

  /home/user/.m2/repository/fr/nice/alea/loterie/1.0.0/loterie-1.0.0.jar
  /home/user/.m2/repository/fr/nice/alea/loterie/1.0.0/loterie-1.0.0.pom
```

Le chemin obtenu, `~/.m2/repository/`*`groupId en répertoires/artifactId/version/`*, est la réponse la plus concrète à la question « c'est quoi le `groupId` ». C'est un **chemin de classement**. Changer le `groupId` après publication d'une version ne déplace pas l'ancienne : elle reste à l'ancienne adresse, et deux coordonnées différentes désignent dès lors deux artefacts distincts aux yeux de Maven.

> **Ne changez pas un `groupId` publié**
>
> Si `fr.nice.alea:loterie:1.0.0` est déjà chez vos consommateurs, passer à `fr.nice:loterie:1.0.0` crée un second artefact. Les dépendances existantes continuent de tirer l'ancien, dont vous ne corrigez plus rien. Le renommage se fait avec une nouvelle version et une procédure de migration — jamais dans l'ombre d'un commit « nettoyage ».

13.8 Choisir son `groupId` : la méthode en cinq questions

> Le `groupId` est le seul des trois champs dont le mauvais choix se paie des années : l'`artifactId` se renomme avec un alias, la `version` avance toute seule, un `groupId` publié ne se corrige pas. Cinq questions, dans cet ordre, et la décision se prend en dix minutes.

> **Question 1 — Contrôlez-vous un nom de domaine ?**
>
> Si oui, la réponse est **le domaine, à l'envers** : `nice.fr` devient `fr.nice`, `example.com` devient `com.example`. C'est la convention de la documentation Maven, celle du tutoriel Java (« Companies use their reversed Internet domain name to begin their package names ») et celle du registre Central, qui demande une preuve de propriété du domaine (enregistrement DNS `TXT`) avant d'accepter le namespace. Vous pouvez ensuite créer autant de sous-groupes que voulu, sans nouvelle autorisation : `fr.nice.alea`, `fr.nice.alea.jeu`, `fr.nice.tools`.

Question 2 — Sinon, avez-vous un compte de forge ?

Pas de domaine ? Le registre Central accepte un `groupId` rattaché à une plateforme de code, vérifiable par la simple création d'un dépôt temporaire. Les formes supportées sont fixées :

Plateforme	`groupId` à utiliser
GitHub	`io.github.votre-nom-utilisateur`
GitLab	`io.gitlab.votre-nom-utilisateur`
Bitbucket	`io.bitbucket.votre-nom-utilisateur`
Gitee	`io.gitee.votre-nom-utilisateur`
SourceForge	`io.sourceforge.votre-nom-utilisateur`

C'est la solution la plus fréquente pour un projet personnel : `io.github.dupont.loterie` s'obtient en quelques minutes, sans domaine à acheter. Si vous vous connectez à Central avec votre compte GitHub, le namespace `io.github.votre-nom` est même accordé automatiquement.

Question 3 — Ce nom restera-t-il disponible ?

Un domaine se perd, un compte de forge se renomme, une entreprise change de marque. Le `groupId` est **permanent par construction** : Central ne supprime pas un artefact publié, et des milliers de projets peuvent en dépendre. Deux précautions : préférez le nom de l'organisation à celui d'un produit qui sera renommé ; et n'utilisez un sous-groupe lié à une personne (`fr.nice.dupont`) que si ce rattachement est assumé durablement.

Question 4 — Le nom est-il un identifiant Java légal ?

Maven accepte `fr-nice-alea` et `fr.nice.2026` ; Java ne les accepte pas tels quels comme packages. Le tutoriel Java donne la règle de conversion : ajouter un souligné. Résultat des essais réels de compilation ci-dessous.

PROGRAMME — HUIT NOMS DE PACKAGE DÉCLARÉS PUIS COMPILÉS (JDK 25)

```
// un fichier par package, chacun déclaré puis compilé séparément
package org.example.hyphenated_name;   // domaine hyphenated-name.example.org
package com.example._123name;          // domaine 123name.example.com
package int_.example;                  // domaine example.int
package fr.nice.alea.jeu;              // forme normale
package org.example.int;               // mot-clé réservé
package com.example.123name;           // segment commençant par un chiffre
package fr.nice.class;                 // mot-clé « class »
package fr.nice.9x;                    // segment commençant par un chiffre
```

SORTIE RÉELLE — `JAVAC -D OUT`

```
  COMPILE   org.example.hyphenated_name
  COMPILE   com.example._123name
  COMPILE   int_.example
  COMPILE   fr.nice.alea.jeu
  REFUSÉ    org.example.int      -> error: <identifier> expected
  REFUSÉ    com.example.123name  -> error: ';' expected
  REFUSÉ    fr.nice.class        -> error: <identifier> expected
  REFUSÉ    fr.nice.9x           -> error: ';' expected
```

Deux noms, deux destins : `fr-nice-alea` est un `groupId` parfaitement valide et un package impossible ; `fr.nice.alea` sert aux deux. C'est le meilleur argument pour choisir un `groupId` qui soit déjà un nom de package légal — la documentation Central le confirme d'ailleurs : elle autorise les tirets dans le `groupId`, mais précise qu'il faut alors garder un nom de package différent.

Question 5 — Simple ou composé ?

Un mot unique est encore possible techniquement, mais Central prévient : « it will be difficult to get a new single word group ID approved ». Les groupes historiques à un seul mot (`commons-io`, `junit`) sont des vestiges. Pour tout projet neuf : au moins deux segments, dont un domaine ou une forge. Le troisième segment sert à séparer les projets d'une même structure.

La grille de décision

Situation	`groupId` recommandé	Pourquoi
Entreprise avec domaine	`fr.nice.alea`	Domaine inversé + nom d'application. Sous-groupes libres ensuite.
Particulier, sans domaine	`io.github.dupont`	Vérifiable immédiatement, gratuit, accepté par Central.
Projet libre hébergé sur GitLab	`io.gitlab.monprojet`	Même mécanisme, preuve par dépôt temporaire.
Bibliothèque destinée à être publiée	`io.github.dupont.loterie`	Le sous-groupe réserve la place pour les autres modules du projet.
Projet interne, jamais publié	`local.dupont` ou le domaine de l'entreprise	Rien ne sort du dépôt local : le seul impératif est l'unicité sur la machine et la lisibilité.
Module d'un projet multi-modules	`fr.nice.alea.jeu`	Maven conseille d'ajouter un segment par module : `org.apache.maven`, `org.apache.maven.plugins`.

Nommer : les dix règles à appliquer

#	Règle	Concrètement
1	Domaine inversé, en minuscules	`example.com` → `com.example`. Jamais `com.Example`.
2	Un préfixe réservé est interdit	N'utilisez jamais `java.`, `javax.` ni `jdk.` : ces espaces appartiennent au JDK.
3	Pas de nom de projet seul	`loterie` comme `groupId` n'identifie personne — c'est un `artifactId`, pas un groupe.
4	ASCII strict	`fr.nice.aléa` est refusé par Maven : `does not match a valid id pattern`.
5	Segments sans espace ni symbole	Refusés, mesurés : espace, `+`, `*`.
6	Pas de tiret si vous voulez le réutiliser comme package	`fr-nice-alea` passe Maven, pas Java. Choisissez `fr.nice.alea`.
7	Pas de segment commençant par un chiffre	`fr.nice.9x` est accepté par Maven et refusé par `javac`.
8	Pas de mot-clé Java	`int`, `class`, `new`, `record`… : refusés comme identifiant.
9	Stable et impersonnel	Le nom de l'organisation, pas le pseudonyme du moment ni le nom du produit.
10	Cohérent avec le package	Le premier choix reste d'utiliser le même nom : moins de conversions, moins d'erreurs.

FICHIER — LES DEUX CONVENTIONS DE CONVERSION, SI LE DOMAINE CONTIENT UN CARACTÈRE GÊNANT

```
hyphenated-name.example.org  ->  groupId : org.example.hyphenated-name
                                 package : org.example.hyphenated_name
123name.example.com          ->  groupId : com.example.123name
                                 package : com.example._123name
```

COMMENTAIRE

```
le groupId s'écrit comme le domaine, exactement (tirets conservés) :
Central le recommande explicitement — « the groupId should reverse the domain
name exactly » — car il sert d'adresse, pas de nom Java.
le package ajoute un souligné, selon le tutoriel Java « Legalizing Package Names ».
```

13.9 Le projet de la leçon, terminé

Voilà le projet complet, celui qui justifie tout ce chapitre : un tirage sans remise, reproductible, avec la graine en paramètre. Le `groupId` est `fr.nice.alea`, l'`artifactId` `loterie`, le package identique au `groupId`.

FICHIER — `SRC/MAIN/JAVA/FR/NICE/ALEA/LOTERIE.JAVA`

```
package fr.nice.alea;

import java.util.*;
import java.util.random.RandomGenerator;

public class Loterie {
    static List<Integer> tirage(int n, int k, RandomGenerator g) {
        if (k < 0 || k > n) throw new IllegalArgumentException(k + " numéros sur " + n);
        List<Integer> boules = new ArrayList<>();
        for (int i = 1; i <= n; i++) boules.add(i);
        Collections.shuffle(boules, g);
        return List.copyOf(boules.subList(0, k));
    }

    public static void main(String[] args) {
        RandomGenerator g = new Random(2026);
        System.out.println("tirage 3 parmi 10, graine 2026 : " + tirage(10, 3, g));
        System.out.println("tirage 5 parmi 49, graine 2026 : " + tirage(49, 5, g));
    }
}
```

SORTIE RÉELLE — DEUX EXÉCUTIONS SUCCESSIVES

```
$ java -cp target/classes fr.nice.alea.Loterie
tirage 3 parmi 10, graine 2026 : [1, 9, 2]
tirage 5 parmi 49, graine 2026 : [37, 42, 39, 43, 16]

$ java -cp target/classes fr.nice.alea.Loterie
tirage 3 parmi 10, graine 2026 : [1, 9, 2]
tirage 5 parmi 49, graine 2026 : [37, 42, 39, 43, 16]
```

Les deux exécutions sont identiques : c'est le chapitre 7 en action. Remplacez `new Random(2026)` par `new SecureRandom()` et les deux lignes diffèrent — c'est le chapitre 9. Le projet, lui, ne change pas : c'est précisément ce qu'apporte une structure correcte.

Réponse courte

Le **`groupId`** est l'identifiant du groupe auquel appartient votre projet — en pratique, votre organisation, écrite en nom de domaine inversé. Il forme, avec l'`artifactId` (le nom du projet) et la `version`, l'adresse unique de votre code : `fr.nice.alea` + `loterie` + `1.0.0`. Maven s'en sert comme chemin de rangement dans le dépôt — `~/.m2/repository/fr/nice/alea/loterie/1.0.0/` — et la convention veut qu'il soit aussi le nom de package Java du projet. Minuscules, points, pas d'accent, pas d'espace : `fr.nice.aléa` est refusé, `fr.nice.alea` accepté.

13.10 Fiche mémo : créer un projet

Champ	Valeur pour ce projet	Règle
`groupId`	`fr.nice.alea`	Domaine inversé que vous contrôlez, minuscules, points. Accepte `-`, `_`, chiffres ; refuse accents, espaces, `+`, `*`.
`artifactId`	`loterie`	Minuscules, chiffres, tirets. Pas de point. Sert de nom au `.jar`.
`version`	`1.0.0`	SemVer : majeur.mineur.correctif. `1.0.0-SNAPSHOT` pour un état intermédiaire.
`package`	`fr.nice.alea`	Par défaut le `groupId`. Peut être plus fin (`fr.nice.alea.jeu`), jamais plus grossier.

Atelier — votre projet, de zéro à la publication

Vingt minutes. À la fin, votre code est réutilisable depuis n'importe quel autre projet de la machine.

1. Choisissez un `groupId` qui vous appartient réellement (domaine, nom d'organisation, ou à défaut `io.github.`*`votre-pseudo`*).
2. Générez le projet avec l'archétype `quickstart`, en vérifiant la ligne `packageInPathFormat` dans la sortie.
3. Corrigez `maven.compiler.release` à `21` et JUnit à `4.13.2`, puis `mvn clean package`.
4. Ajoutez la classe `Loterie` ci-dessus, exécutez-la deux fois, vérifiez que les tirages sont identiques.
5. `mvn install`, puis vérifiez le chemin du `.jar` dans `~/.m2/repository` : les points du `groupId` doivent être devenus des répertoires.
6. Dernière question, celle qui compte : ce `groupId` serait-il acceptable sur Maven Central ? La documentation prévient que « it will be difficult to get a new single word group ID approved » — un domaine inversé évite le sujet.